

Phase 1 — LangChain LCEL

Cinq concepts essentiels + cinq exercices pour aller plus loin

Cours PyTutor2

Résumé

Ce document couvre les cinq briques fondamentales de la phase 1 du cours PyTutor2 : les `ChatPromptTemplate`, la sortie typée avec Pydantic et `with_structured_output`, la composition LCEL avec `|` et `.partial()`, l'exécution parallèle avec `RunnableParallel`, et le streaming token par token avec `.stream()`. Chaque concept est traité avec un énoncé, une architecture illustrée en TikZ, le code complet, des explications pas à pas et la vérification dans LangSmith. Une seconde partie « *Pour aller plus loin* » prolonge avec cinq exercices avancés : parallélisation à 3 branches, `.batch()` sur plusieurs inputs, extension d'un schéma Pydantic, router avec `RunnableBranch`, et exécution `async`.

Table des matières

I	Exercices	2
1	ChatPromptTemplate : structurer un prompt en messages paramétrés	2
1.1	Solution	2
1.1.1	Architecture	2
1.1.2	Le code complet	3
1.1.3	Explications pas à pas	3
1.2	Vérification dans LangSmith	5
2	Sortie typée : schémas Pydantic et with_structured_output	6
2.1	Solution	6
2.1.1	Architecture	6
2.1.2	Le code complet	6
2.1.3	Explications pas à pas	7
2.2	Vérification dans LangSmith	9
3	LCEL : composition avec et figement de variables avec .partial()	10
3.1	Solution	11
3.1.1	Architecture	11
3.1.2	Le code complet	11
3.1.3	Explications pas à pas	12
3.2	Vérification dans LangSmith	13
4	RunnableParallel : exécuter plusieurs chaînes en concurrence	15
4.1	Solution	15
4.1.1	Architecture	15
4.1.2	Le code complet	16
4.1.3	Explications pas à pas	16
4.1.4	Exécution et observation	18
4.2	Vérification dans LangSmith	18

5	Streaming token par token avec <code>.stream()</code>	19
5.1	Solution	20
5.1.1	Architecture	20
5.1.2	Le code complet	20
5.1.3	Explications pas à pas	21
5.2	Vérification dans LangSmith	22
II	Pour aller plus loin	23
6	Multi-niveaux 3× : parallèle vs séquentiel	23
6.1	Solution	24
6.1.1	Architecture	24
6.1.2	Le code complet	24
6.1.3	Explications pas à pas	25
6.2	Vérification dans LangSmith	25
7	<code>.batch()</code> vs boucle <code>for</code>	26
7.1	Solution	26
7.1.1	Architecture	26
7.1.2	Le code complet	27
7.1.3	Explications pas à pas	27
7.2	Vérification dans LangSmith	28
8	Schéma Pydantic enrichi avec un champ <code>analogy</code>	29
8.1	Solution	29
8.1.1	Architecture	29
8.1.2	Le code complet	29
8.1.3	Explications pas à pas	30
8.2	Vérification dans LangSmith	31
9	Router avec <code>RunnableBranch</code> et classifieur LLM	32
9.1	Solution	32
9.1.1	Architecture	32
9.1.2	Le code complet	33
9.1.3	Explications pas à pas	34
9.2	Vérification dans LangSmith	35
10	Async avec <code>asyncio.gather</code> vs <code>RunnableParallel</code>	36
10.1	Solution	36
10.1.1	Architecture	36
10.1.2	Le code complet	37
10.1.3	Explications pas à pas	37
10.2	Vérification dans LangSmith	39

Première partie

Exercices

1 ChatPromptTemplate : structurer un prompt en messages paramétrés

Exercice 1. ChatPromptTemplate

On veut produire un assistant pédagogique Python dont le **ton et la profondeur** s'adaptent au niveau de l'élève (débutant, intermédiaire, avancé). Le concept à expliquer change à chaque requête, le niveau aussi.

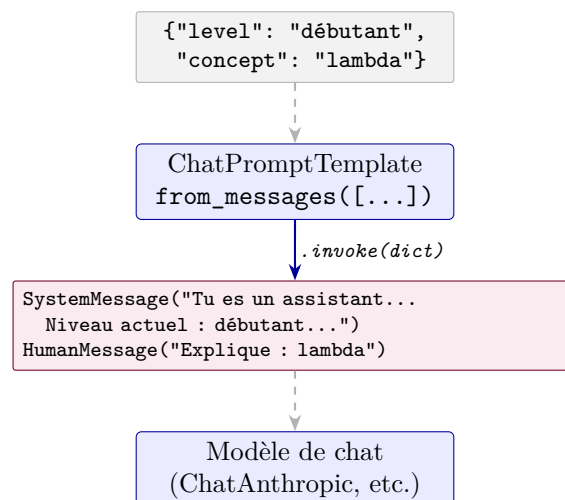
Contrainte : ne pas écrire trois prompts différents, ni concaténer des strings à la main. On veut *un seul* template réutilisable, avec deux variables `{level}` et `{concept}`, qui sera ensuite consommé par n'importe quel modèle compatible LangChain.

Objectifs pédagogiques

- Comprendre pourquoi un prompt moderne est une **liste de messages typés**, pas une chaîne de caractères.
- Maîtriser la séparation **system** / **human** et son rôle pour la qualité des réponses.
- Utiliser les **variables** `{nom}` d'un template et savoir ce qu'elles deviennent au moment du rendu.

1.1 Solution

1.1.1 Architecture



Un `ChatPromptTemplate` est essentiellement une **fonction** `dict → List[BaseMessage]`. Les variables `{level}` et `{concept}` sont substituées dans le contenu de chaque message au moment du rendu.

1.1.2 Le code complet

```
1 from langchain_core.prompts import ChatPromptTemplate
2
3 EXPLAIN_PROMPT = ChatPromptTemplate.from_messages([
4     (
```

```

5     "system",
6     "Tu es un assistant pédagogique pour des élèves apprenant Python. "
7     "Tu adaptes systématiquement ton vocabulaire et tes exemples au "
8     "niveau de l'élève. Niveau actuel : {level}.\n\n"
9     "Règles selon le niveau :\n"
10    " - 'débutant' : pas de jargon, exemples ultra-simples, "
11    "analogies du quotidien, pas plus de 10 lignes de code.\n"
12    " - 'intermédiaire' : tu peux supposer la syntaxe de base "
13    "connue, montre les usages idiomatiques (PEP 8, type hints).\n"
14    " - 'avancé' : montre les détails du modèle objet de Python, "
15    "les subtilités, les cas limites, les implications de performance."
16    ),
17    ("human", "Explique le concept suivant : {concept}"),
18 ])

```

1.1.3 Explications pas à pas

Étape 1 — Pourquoi une *liste de messages* et pas un string

Les modèles de chat modernes (Claude, GPT-4, Gemini, etc.) ne reçoivent pas un prompt monolithique : ils reçoivent une **conversation structurée** sous forme de messages typés. Chaque message a un **role** (**system**, **user/human**, **assistant**) et un contenu. L'API du fournisseur les traite différemment :

- Le **system** fixe la **personnalité** et les règles du jeu, et reçoit le plus de poids dans le respect des instructions.
- Les **human** sont les requêtes de l'utilisateur.
- Les **assistant** sont les réponses précédentes du modèle (utile pour du multi-tour, on les verra plus tard).

*Mettre les règles dans le **system** et la requête dans le **human** n'est pas cosmétique : les LLM modernes obéissent nettement mieux à une instruction présente dans le message système qu'à la même instruction noyée dans le message utilisateur. Si le ton de ton assistant dérive, c'est souvent que tu as mis tes règles au mauvais endroit.*

Étape 2 — La syntaxe `from_messages(...)`

`from_messages` accepte une liste de **tuples** de la forme (**role**, **contenu**) :

<code>("system", "...")</code>	devient un <code>SystemMessagePromptTemplate</code>
<code>("human", "...")</code>	devient un <code>HumanMessagePromptTemplate</code>
<code>("ai", "...")</code>	devient un <code>AIMessagePromptTemplate</code>
<code>("placeholder", "{x}")</code>	insère une liste de messages stockée dans <code>x</code>

LangChain transforme automatiquement chaque tuple en un objet `PromptTemplate` de la bonne classe, et l'ensemble compose un `ChatPromptTemplate`.

Étape 3 — Les variables `{nom}`

Toutes les accolades non-doublées dans une chaîne de message sont considérées comme des **variables** :

- `{concept}` dans le **human** attend une clé `concept` dans le dict d'entrée.
- `{level}` dans le **system** attend une clé `level`.
- Si une clé manque au moment du rendu, LangChain lève une `KeyError` explicite.

*Si tu veux écrire une vraie accolade dans le prompt (par exemple, demander au modèle de répondre en JSON), il faut la **doubler** : `{{` pour un `{` littéral, `}}` pour un `}`. C'est le piège classique qui transforme un message en `KeyError: 'nom_de_clé_inattendu'` dès qu'on parle de structures JSON dans le prompt.*

Étape 4 — Le rendu via `.invoke()` ou `.format_messages()`



Deux façons d'appeler le template :

1. `EXPLAIN_PROMPT.invoke({"level": "débutant", "concept": "lambda"})` renvoie un `ChatPromptValue` (qu'on passe ensuite au modèle). C'est la voie LCEL, utilisée quand le template est dans une chaîne `prompt | model | ...`.
2. `EXPLAIN_PROMPT.format_messages(...)` renvoie directement la liste de `BaseMessage`. Pratique pour déboguer ou pour passer les messages à une API tierce.

*.format_messages(...) est le print du prompt engineer. Avant de te demander pourquoi le modèle répond bizarrement, imprime ce que **tu lui as réellement envoyé** — souvent l'erreur est là.*

Sortie de l'exécution

```
=====
DÉMO 1 - Explication structurée : 'les générateurs Python' (intermédiaire)
=====
```

Intuition :

Un générateur est une fonction qui produit une séquence de valeurs une à la fois, au lieu de tout calculer et retourner d'un coup. Imagine une usine qui fabrique des objets à la demande plutôt que de les stocker tous en entrepôt : elle économise de l'espace et de l'énergie. En Python, les générateurs utilisent le mot-clé `yield` pour « pausable » et reprendre l'exécution, ce qui les rend très efficaces pour traiter de grandes quantités de données.

Code :

```
# Fonction classique : calcule et retourne TOUT d'un coup
def nombres_classique(n: int) -> list:
    resultat = []
    for i in range(n):
        resultat.append(i ** 2)
    return resultat

# Générateur : produit les valeurs UNE À LA FOIS
def nombres_générateur(n: int):
    for i in range(n):
        yield i ** 2 # Pause ici, attend qu'on demande la prochaine valeur

# Utilisation
print("Classique :", nombres_classique(5)) # [0, 1, 4, 9, 16]

print("Générateur :")
gen = nombres_générateur(5)
print(next(gen)) # 0 (première valeur)
print(next(gen)) # 1 (deuxième valeur)

# Ou avec une boucle (plus courant)
for valeur in nombres_générateur(5):
    print(valeur) # Affiche 0, 1, 4, 9, 16 une à une
```

Piège classique :

Oublier que les générateurs ne peuvent être parcourus qu'une seule fois. Une fois épuisés, ils ne produisent plus de valeurs. Si tu essaies de les réutiliser, tu dois en créer une nouvelle instance. Aussi, confondre `yield` avec `return` : `yield` pause et reprend, tandis que `return` termine définitivement la fonction.

Questions de compréhension :

1. Quelle est la différence principale entre une fonction classique qui retourne une liste et un générateur ?
→ Une fonction classique calcule et retourne toutes les valeurs à la fois en mémoire, tandis qu'un générateur

produit les valeurs une à la fois à la demande, ce qui économise la mémoire.

2. Comment appelle-t-on le mot-clé qui permet à une fonction de devenir un générateur ?
→ Le mot-clé `yield`. Quand une fonction contient `yield`, elle devient un générateur au lieu d'une fonction classique.
3. Pourquoi les générateurs sont-ils particulièrement utiles pour traiter des fichiers très volumineux ou des flux de données infinis ?
→ Parce qu'ils ne chargent pas toutes les données en mémoire à la fois, mais les traitent progressivement. Cela permet de traiter des données plus grandes que la RAM disponible et de réagir immédiatement aux premières valeurs sans attendre le calcul complet.

1.2 Vérification dans LangSmith

Comment voir le prompt rendu

Quand le template est utilisé dans une chaîne et que LangSmith est activé, chaque appel au modèle produit un run `ChatAnthropic` (ou équivalent) dans lequel l'onglet *Input* contient **exactement les messages rendus** :

```
RunnableSequence
    ChatPromptTemplate
        Input : {level: "débutant", concept: "lambda"}
        Output : [SystemMessage(...), HumanMessage(...)]
    ChatAnthropic
        Input : la liste de messages ci-dessus
```

Les trois choses à regarder

1. L'onglet *Input* du nœud `ChatPromptTemplate` : tu vois le dict d'entrée brut. C'est là que tu repères une variable manquante ou mal typée.
2. L'onglet *Output* du même nœud : la liste complète des messages, variables substituées, prêts à partir au LLM. C'est la **vérité terrain** de ce que le modèle reçoit.
3. L'onglet *Input* du nœud `ChatAnthropic` : en principe identique à l'output du template ci-dessus. Si tu y vois autre chose, c'est qu'un `Runnable` intermédiaire a modifié les messages.

Change la valeur de `level` entre "débutant" et "avancé" pour le même concept. Dans LangSmith, tu vois la même structure de messages, seul le contenu du `system` diffère. C'est la visualisation directe de la paramétrisation par template.

Récapitulatif

Ce qu'il faut retenir :

1. Un prompt de chat est une **liste de messages typés**, pas un `string`. Le rôle (`system/human/ai`) change radicalement la façon dont le modèle traite le contenu.
2. `ChatPromptTemplate.from_messages([...])` est la voie idiomatique pour construire ce prompt. Les tuples (`role, content`) sont la forme la plus lisible.
3. Les **variables** `{nom}` dans le contenu deviennent des clés attendues du dict d'entrée. Doubler les accolades pour les accolades littérales.
4. `prompt.invoke(dict)` renvoie un `ChatPromptValue` pour la chaîne `LCEL` ; `prompt.format_messages(dict)` renvoie la liste brute, indispensable pour le débogage.
5. Mettre les règles dans le `system`, la requête dans le `human`. C'est l'erreur #1 qui fait dériver le ton d'un assistant.

2 Sortie typée : schémas Pydantic et with_structured_output

Exercice 2. Sortie typée avec Pydantic

Un assistant pédagogique ne doit pas répondre par un pavé de texte markdown. On veut un objet manipulable : une *explication structurée* contenant l'intuition, un exemple de code, un piège fréquent et une liste de questions de compréhension. Chaque champ a une sémantique précise, chaque question a sa réponse attendue.

Contrainte : la réponse doit être *immédiatement utilisable* en Python, sans étape de parsing fragile. `result.intuition` doit fonctionner ; `result.comprehension_questions[0].expected_answer` aussi.

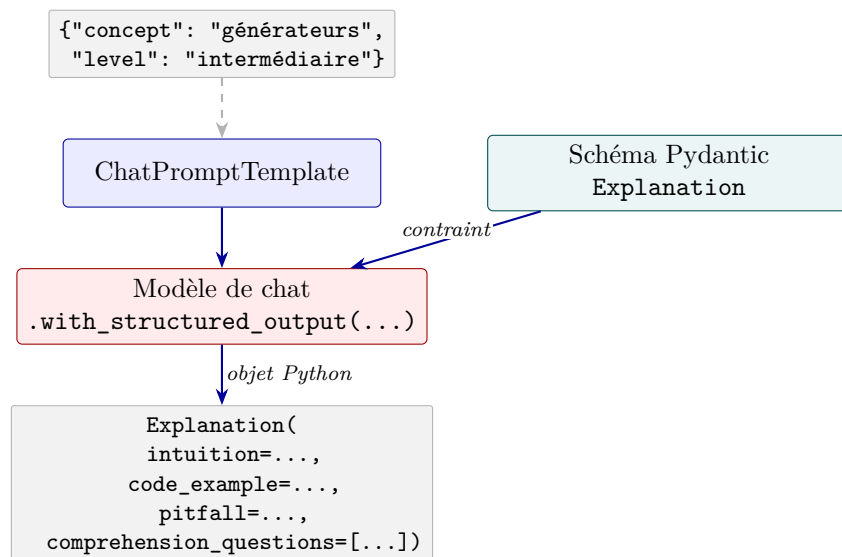
Objectifs pédagogiques

- Comprendre pourquoi exiger une **sortie structurée** d'un LLM change la nature du code qu'on écrit autour.
- Décrire un schéma de réponse avec `pydantic.BaseModel` et `Field(description=...)`.
- Maîtriser `model.with_structured_output(Schema)` et savoir ce qui se passe **sous le capot** (tool-calling).
- Reconnaître les cas où il faut *imbriquer* des sous-schémas.

2.1 Solution

2.1.1 Architecture

Vue d'ensemble



Le schéma Pydantic est **injecté dans l'appel** au modèle. Le SDK Anthropic l'expose comme un *tool* que Claude doit appeler ; la réponse en JSON est ensuite désérialisée vers un vrai objet Python.

2.1.2 Le code complet

```
1 from pydantic import BaseModel, Field
2
3 class ComprehensionQuestion(BaseModel):
4     """Une question de compréhension pour vérifier l'acquisition."""
5
6     question: str = Field(description="La question posée à l'élève.")
```

```

7     expected_answer: str = Field(
8         description="La réponse attendue, en une à deux phrases."
9     )
10
11
12 class Explanation(BaseModel):
13     """Réponse pédagogique complète sur un concept Python."""
14
15     concept: str = Field(description="Le nom du concept expliqué.")
16     intuition: str = Field(
17         description=(
18             "Explication intuitive en exactement 3 phrases. "
19             "Évite le jargon, vise la clarté pour l'élève."
20         )
21     )
22     code_example: str = Field(
23         description=(
24             "Un exemple de code Python minimal et commenté qui illustre "
25             "le concept. Préfère des noms de variables parlants."
26         )
27     )
28     pitfall: str = Field(
29         description="Un piège classique que rencontrent les élèves sur ce concept."
30     )
31     comprehension_questions: list[ComprehensionQuestion] = Field(
32         description="Exactement 3 questions de compréhension, de difficulté croissante."
33     )
34
35
36 # Utilisation dans une chaîne LCEL
37 def explain_chain_for_level(level: str):
38     model = get_model().with_structured_output(Explanation)
39     return EXPLAIN_PROMPT.partial(level=level) | model

```

2.1.3 Explications pas à pas

Étape 1 — Pourquoi structurer la sortie

Sans schéma, un LLM répond en texte libre. Le code applicatif doit alors *parser* cette réponse : regex, `json.loads`, conventions de markdown... Chaque appel devient un pari sur le fait que le modèle respectera bien le format demandé. En production, ce pari échoue régulièrement : virgule manquante, guillemets inversés, balise oubliée.

*Faire écrire le format au modèle est moins fiable que le lui imposer. La sortie structurée renverse la responsabilité : ce n'est plus le modèle qui « doit essayer de respecter » un format ; c'est l'API qui **refuse** toute sortie qui ne le respecte pas. Le code applicatif récupère un objet typé, point.*

Étape 2 — Le schéma comme *contrat* de réponse

Un schéma Pydantic n'est pas un simple type : c'est une description **lisible par le LLM**. Quand on appelle `with_structured_output(Explanation)`, LangChain transforme la classe en une *spécification de tool* qui contient :

- Le nom du schéma (`Explanation`).
- Le `docstring` de la classe (« Réponse pédagogique complète sur un concept Python »).
- Pour chaque champ : son nom, son type, et surtout sa `description=`.

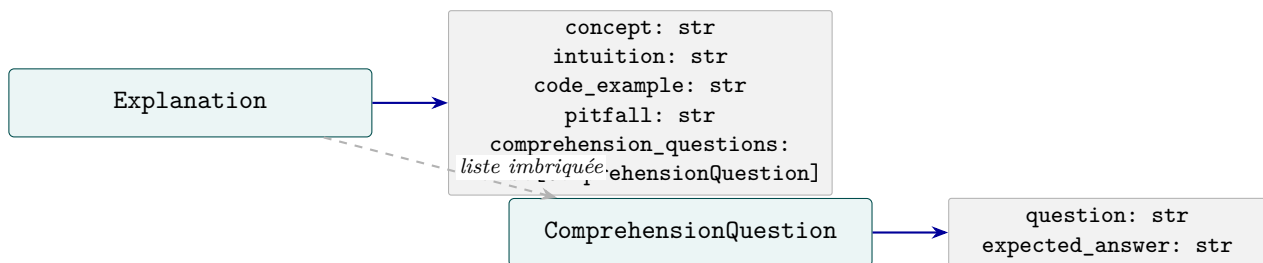
La *description* de chaque *Field* est une *partie du prompt*. Le modèle la lit pour décider quoi mettre dans le champ. Soigne-la au moins autant que ton message système.

Étape 3 — Les types qui parlent au modèle

<code>str</code>	champ texte libre
<code>int, float</code>	nombre, validation automatique
<code>bool</code>	vrai / faux
<code>Literal["a", "b", "c"]</code>	ensemble fermé de valeurs (= énumération)
<code>list[X]</code>	liste typée (peut être un sous-schéma)
<code>X None</code>	champ optionnel
<code>BaseModel imbriqué</code>	sous-objet typé (cf. <code>ComprehensionQuestion</code>)

Tous ces types sont traduits en JSON Schema. Le LLM reçoit ce schéma et sait, par exemple, qu'un champ `Literal` n'a que 3 valeurs possibles — il ne pourra *physiquement pas* en renvoyer une quatrième.

Étape 4 — L'imbrication de schémas



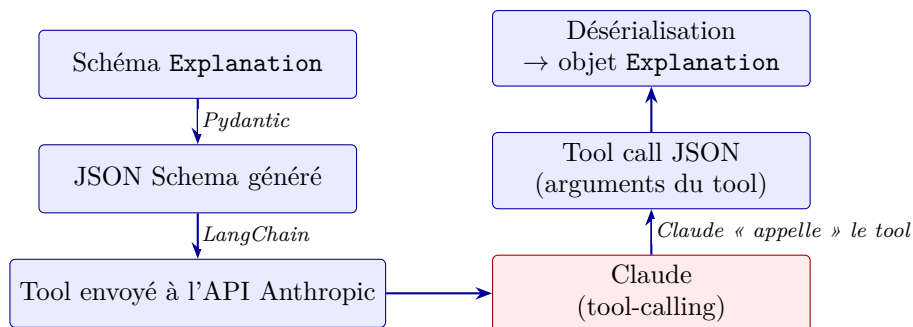
`Explanation` contient une `list[ComprehensionQuestion]`. Chaque question dans la liste est elle-même un objet typé, qu'on peut itérer en Python :

```

1 for q in result.comprehension_questions:
2     print(q.question)           # str
3     print(q.expected_answer)    # str
  
```

Les sous-schémas sont la voie idiomatique pour modéliser des listes d'objets (questions, étapes, sources, citations...). Évite de stocker du JSON sérialisé dans un champ `str` — tu perds la validation et l'auto-complétion.

Étape 5 — Ce qui se passe sous le capot (tool-calling)



`with_structured_output` n'invente pas un parseur de réponse texte. Il déclare un *tool* virtuel auprès du modèle, dont les arguments **sont** les champs du schéma. Claude est alors quasi-forcé d'« appeler » ce tool ; les arguments arrivent en JSON valide, validés par Pydantic, et reconvertis en objet Python.

Si tu utilises un modèle qui ne supporte pas le tool-calling natif (certains modèles open-source), `with_structured_output` retombe sur un parsing texte plus fragile. Vérifie toujours que le modèle ciblé supporte les tools — c’est le cas de tous les Claude messages, de GPT-4, et de Gemini.

Étape 6 — L’objet de sortie côté Python

```
1 chain = explain_chain_for_level("intermédiaire")
2 result: Explanation = chain.invoke({"concept": "les générateurs Python"})
3
4 # result est un VRAI objet Python.
5 # - PyCharm autocomplète result.intuition, result.pitfall, etc.
6 # - mypy / Pyright valident les accès.
7 # - On peut le sérialiser avec result.model_dump_json(indent=2).
8 print(result.intuition)
9 for q in result.comprehension_questions:
10     print(f"- {q.question}")
```

Le `result: Explanation` en annotation de type est important. Sans elle, l’IDE pense que `result` est de type `Any` et tu perds l’auto-complétion sur les champs. C’est le cadeau de la sortie structurée — sois sûr de l’utiliser.

Sortie de l’exécution

```
=====
DÉMO 2 - RunnableParallel : 2 niveaux en concurrence
=====
— Pour un DÉBUTANT —
Un générateur est une fonction spéciale qui produit des valeurs une à la fois, au lieu de tout créer d’un coup.
Imagine une usine qui fabrique des bonbons : au lieu de fabriquer 1000 bonbons et les stocker tous ensemble,
elle en fabrique un seul quand tu en demandes un. C’est plus économe en mémoire et plus rapide au démarrage.
— Pour un INTERMÉDIAIRE —
Un générateur est une fonction qui produit une séquence de valeurs une à la fois, au lieu de tout calculer et
retourner d’un coup. Imagine une usine qui fabrique des objets à la demande plutôt que de les stocker tous
en entrepôt : c’est plus efficace en mémoire. Quand tu appelles un générateur, il te donne la prochaine valeur
chaque fois que tu la demandes, en se « souvenant » où il en était.
```

2.2 Vérification dans LangSmith

L’arbre d’exécution attendu

```
RunnableSequence
  ChatPromptTemplate
    ChatAnthropic (structured_output=Explanation)
      Input : List[BaseMessage]
      Output : Explanation(...)
      Tool call : Explanation({...})
```

Les trois choses à regarder

1. **Onglet *Tool calls*** du nœud `ChatAnthropic` : tu y vois le tool `Explanation` déclaré, avec son JSON Schema complet (les types et les `description=` de chaque champ). C’est la **preuve** que ton schéma est bien transmis au modèle.
2. **Onglet *Output*** : la réponse brute du modèle est un tool-call dont les `args` contiennent les champs du schéma. `LangChain` la transforme automatiquement en objet `Explanation`.

3. **Onglet *Costs*** : noter que la sortie structurée coûte légèrement plus cher en tokens d'entrée (le schéma est envoyé) mais souvent moins en tokens de sortie (le modèle ne génère pas de préambule du genre « Voici votre réponse en JSON... »).

*Modifie une **description**= d'un champ et relance. Tu vois dans LangSmith que la nouvelle description est immédiatement présente dans le tool envoyé au LLM, et que le contenu du champ change conformément. C'est le prompt engineering appliqué directement au schéma de sortie.*

Récapitulatif

Ce qu'il faut retenir :

1. **Sortie structurée** = on impose au LLM un schéma de réponse, on récupère un objet Python typé, plus de parsing fragile.
2. `pydantic.BaseModel + Field(description=...)` est la voie standard. Les descriptions sont lues par le modèle, elles font partie du prompt.
3. `model.with_structured_output(Schema)` fonctionne via le **tool-calling** natif des API modernes — donc fiable, sans ré-essais ni parseur custom.
4. Types puissants à connaître : `Literal[...]` pour les énumérations, `BaseModel` imbriqué pour les sous-objets, `list[Schema]` pour les listes typées.
5. Annoter `result` : `MonSchema` dans le code applicatif pour bénéficier de l'auto-complétion et de la vérification statique.

3 LCEL : composition avec | et figement de variables avec .partial()

Exercice 3. Composition LCEL avec | et .partial()

On dispose d'un prompt unique `EXPLAIN_PROMPT` qui prend deux variables `{level}` et `{concept}`. On veut produire **trois chaînes spécialisées** (débutant, intermédiaire, avancé), chacune n'attendant plus que `{concept}` en entrée — le niveau étant déjà figé.

Contrainte : pas de duplication du prompt, pas de fonction `if/elif` qui réécrit la chaîne à chaque appel. La spécialisation doit se faire **à la construction**, et chaque chaîne produite doit rester un `Runnable` composable, traçable et streamable.

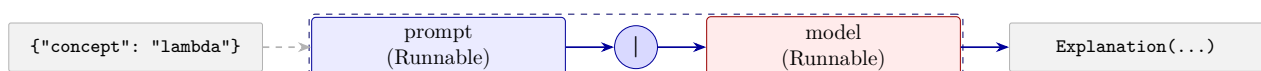
Objectifs pédagogiques

- Comprendre LCEL (*LangChain Expression Language*) comme une **algèbre de composition** de `Runnable`.
- Maîtriser l'opérateur `|` et le type qu'il produit.
- Utiliser `.partial()` pour figer une variable d'un prompt et produire des **chaînes spécialisées sans dupliquer de code**.
- Reconnaître le pattern *fabrique* (*factory*) pour construire dynamiquement des chaînes paramétrées.

3.1 Solution

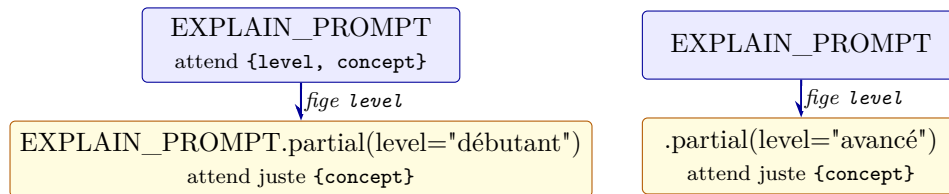
3.1.1 Architecture

La composition par `|`



prompt | model crée un nouveau Runnable qui chaîne les deux. La sortie du gauche devient l'entrée du droit. C'est typé, c'est composable, c'est traçable.

Le figement par .partial()



.partial() ne mute pas le prompt d'origine : il renvoie une **nouvelle** version partiellement renseignée. C'est l'équivalent de `functools.partial` pour les prompts.

3.1.2 Le code complet

```
1 from langchain_core.prompts import ChatPromptTemplate
2 from config import get_model
3
4 EXPLAIN_PROMPT = ChatPromptTemplate.from_messages([
5     ("system", "Tu es un assistant pédagogique Python. "
6         "Niveau actuel : {level}."),
7     ("human", "Explique : {concept}"),
8 ])
9
10 def explain_chain_for_level(level: str):
11     """Renvoie une chaîne dont le niveau est figé.
12
13     `.partial(level=...)` crée un nouveau prompt avec une variable fixée.
14     On peut donc construire des chaînes spécialisées sans dupliquer le template.
15     """
16     model = get_model().with_structured_output(Explanation)
17     return EXPLAIN_PROMPT.partial(level=level) | model
18
19
20 # Trois chaînes spécialisées, un seul template
21 beginner_chain = explain_chain_for_level("débutant")
22 intermediate_chain = explain_chain_for_level("intermédiaire")
23 advanced_chain = explain_chain_for_level("avancé")
24
25 # Chaque chaîne n'attend plus que {concept}
26 result = intermediate_chain.invoke({"concept": "les générateurs Python"})
```

3.1.3 Explications pas à pas

Étape 1 — Qu'est-ce qu'un Runnable

LCEL repose sur une **interface unique** : tout objet manipulable est un Runnable. Cette interface garantit :

- `.invoke(input)` — exécution synchrone, un input.
- `.batch([inputs])` — exécution synchrone, plusieurs inputs.
- `.stream(input)` — itérateur, pour le streaming.
- `.ainvoke`, `.abatch`, `.astream` — versions asynchrones.

ChatPromptTemplate, un modèle, un parseur, un schéma — tous sont des Runnable. C'est ce qui permet de les composer librement.

Un seul protocole pour tout. C'est l'innovation de LCEL par rapport aux versions antérieures de LangChain : au lieu d'avoir des *LLMChain*, *SequentialChain*, *RouterChain*, ..., il n'y a qu'un type composable. Le code applicatif n'a plus à connaître qu'un seul vocabulaire.

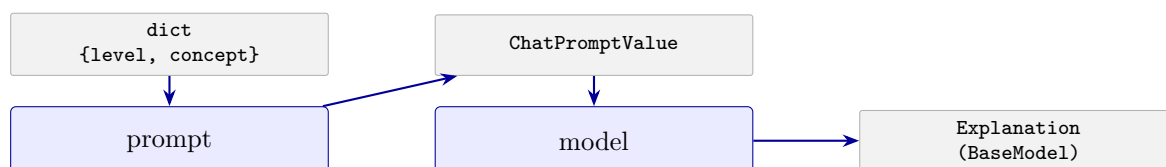
Étape 2 — Le | comme sucre syntaxique

`a | b` appelle `a.__or__(b)`, méthode définie sur la classe de base Runnable. Elle retourne une instance de `RunnableSequence(a, b)`. Mentalement, la transformation est :

$$\text{prompt} \mid \text{model} \quad \equiv \quad \text{RunnableSequence}(\text{prompt}, \text{model})$$

Et `a | b | c` compose en chaîne (associatif à gauche) — équivalent à `RunnableSequence(a, b, c)`. Quand on `.invoke(x)` le résultat, LCEL fait `c.invoke(b.invoke(a.invoke(x)))`.

Étape 3 — Le contrat de types entre maillons



La sortie d'un maillon doit être acceptable par le suivant. Le modèle accepte indifféremment un ChatPromptValue, une liste de messages ou un string — c'est ce qui fait que `prompt | model` marche naturellement.

Le | ne vérifie pas les types à la composition. Un mauvais chaînage (par exemple `model | prompt`) ne plantera qu'au `.invoke()`, avec une erreur souvent obscure. Compose toujours dans l'ordre entrée → traitement → sortie.

Étape 4 — .partial() : la fonction partielle des prompts

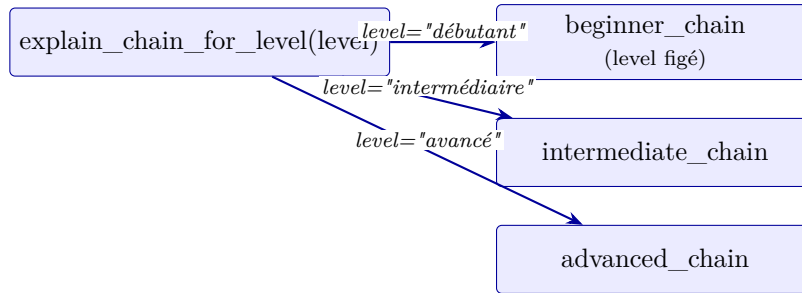
Un ChatPromptTemplate qui attend deux variables {level} et {concept} ressemble à une fonction Python `f(level, concept)`. `partial(level="débutant")` produit une nouvelle fonction `g(concept)` où `level` est figé.

```
1 # Le prompt d'origine attend les deux variables.
2 EXPLAIN_PROMPT.input_variables
3 # >>> ['level', 'concept']
4
5 beginner_prompt = EXPLAIN_PROMPT.partial(level="débutant")
6 beginner_prompt.input_variables
7 # >>> ['concept']
8
9 # On peut désormais invoquer en ne passant que {concept}.
10 beginner_prompt.invoke({"concept": "lambda"})
```

.partial() ne mute pas le prompt original. C'est une opération pure qui renvoie une nouvelle instance. Tu peux donc appeler `EXPLAIN_PROMPT.partial(...)` cent fois sans casser le template de référence.

Étape 5 — Le pattern *fabrique*

`explain_chain_for_level(level)` est une **fabrique de chaînes** : on l'appelle avec un paramètre, elle nous renvoie un `Runnable` prêt à l'emploi, déjà configuré.



Avantages :

- **Pas de duplication** du prompt (un seul source de vérité).
- **Spécialisation à la construction**, pas à l'appel. Une fois construite, la chaîne ne demande plus le niveau à chaque `invoke`.
- **Composabilité conservée** : les chaînes spécialisées peuvent être passées à un `RunnableBranch`, un `RunnableParallel`, etc.

*On vient de remplacer trois fonctions différentes ou trois copies du prompt par **une fabrique paramétrée**. C'est l'esprit LCEL : construire dynamiquement des graphes d'exécution à partir de briques primitives, plutôt que d'écrire du code impératif.*

Sortie de l'exécution

```
=====
DÉMO 3 - Streaming token par token
=====

# Le GIL (Global Interpreter Lock) en Python
Le GIL est un mécanisme de verrouillage global dans l'interpréteur CPython qui empêche l'exécution simultanée de bytecode Python par plusieurs threads. Même sur un processeur multi-cœurs, un seul thread peut exécuter du code Python à la fois. Cela existe pour simplifier la gestion mémoire de CPython et éviter les conditions de course lors de l'accès aux objets Python.

Conséquences pratiques : Le GIL limite les performances des programmes multi-threadés pour les tâches CPU-intensives. Pour contourner ce problème, on peut utiliser le multiprocessing (processus séparés), les asyncio (programmation asynchrone), ou des bibliothèques comme NumPy qui libèrent le GIL. Le GIL n'affecte pas les opérations I/O (réseau, fichiers) car le verrou est libéré pendant ces opérations.
```

3.2 Vérification dans LangSmith

Que vois-tu dans l'arbre

Chaque chaîne produite par la fabrique est un `RunnableSequence`. Quand tu `invoke` `beginner_chain`, LangSmith trace :

```
RunnableSequence (beginner_chain)
  ChatPromptTemplate
    Input : {concept: "lambda"}
    Output : SystemMessage("...Niveau : débutant...")
  ChatAnthropic (structured_output=Explanation)
```

Le `level` n'apparaît plus dans l'`input` du run — il a été absorbé par le `.partial()`. C'est la preuve visuelle que le figement est bien effectif.

Les trois choses à regarder

1. **Le *type* du run racine** : `RunnableSequence`. C'est la matérialisation du `|`.
2. **L'input du `ChatPromptTemplate`** : seulement la (ou les) variable(s) non figée(s). Si tu y vois encore `level`, c'est que ton `.partial()` n'a pas été pris en compte.
3. **L'output du `ChatPromptTemplate`** : les messages complets avec `level` déjà remplacé. La substitution a eu lieu à la construction du template, pas dans le modèle.

Le tracing LangSmith permet de voir la différence entre « variable figée par `.partial()` » et « variable passée à `.invoke()` » : la première est invisible dans l'input du run, la seconde est visible. C'est utile pour déboguer les chaînes paramétrées.

Récapitulatif

Ce qu'il faut retenir :

1. LCEL repose sur **une seule interface** : `Runnable`. Tout objet manipulable (prompt, modèle, parseur, branch, parallel, ...) implémente `.invoke`, `.batch`, `.stream`, et leurs versions `async`.
2. `a | b` crée un `RunnableSequence(a, b)`. C'est purement du sucre syntaxique sur la méthode `Runnable.__or__`.
3. `ChatPromptTemplate.partial(...)` fige une ou plusieurs variables sans muter le template original. La chaîne produite n'attend plus que les variables restantes.
4. **Pattern fabrique** : une fonction `build_xxx(...)` qui renvoie un `Runnable` paramétré est la manière idiomatique de spécialiser à la construction plutôt qu'à l'appel.
5. Dans LangSmith, le `|` apparaît comme un nœud `RunnableSequence`; `.partial()` se voit à l'absence de la variable figée dans l'input du run.

4 RunnableParallel : exécuter plusieurs chaînes en concurrence

Exercice 4. RunnableParallel

On veut générer **deux versions** d'une même explication : une pour les débutants, une pour les intermédiaires. Les deux explications sont **indépendantes** l'une de l'autre (aucune n'a besoin du résultat de l'autre).

Contrainte de performance : chaque appel au LLM prend ~ 1 à 2 secondes. En exécution séquentielle, on attendrait ~ 2 à 4 secondes au total. On veut **ne payer que la latence d'un seul appel**, soit ~ 1 à 2 secondes — et un seul `invoke` côté code applicatif.

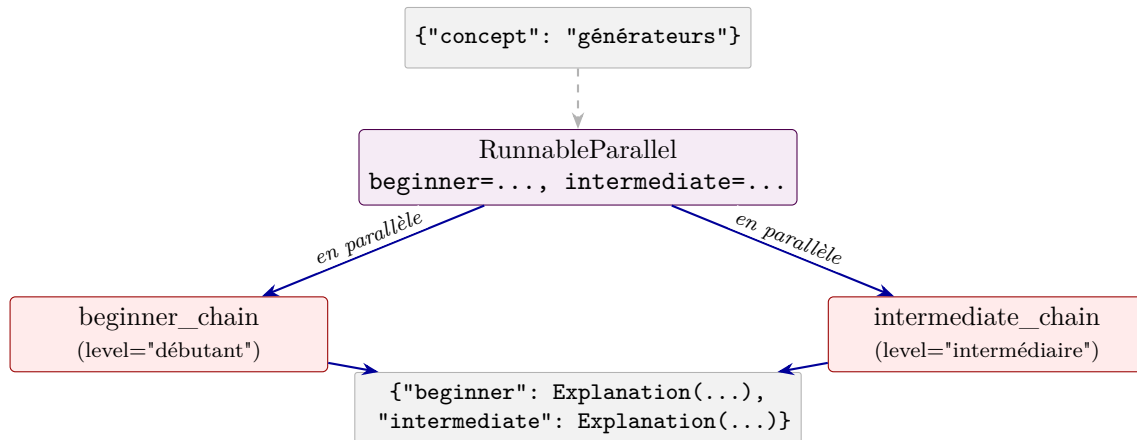
Objectifs pédagogiques

- Comprendre comment LCEL *parallélise* des chaînes indépendantes avec `RunnableParallel`.
- Quantifier le gain de latence par rapport à une boucle séquentielle.
- Distinguer `RunnableParallel` de `RunnablePassthrough.assign` (qui paraît proche mais a une sémantique différente).
- Repérer la structure « en éventail » dans LangSmith.

4.1 Solution

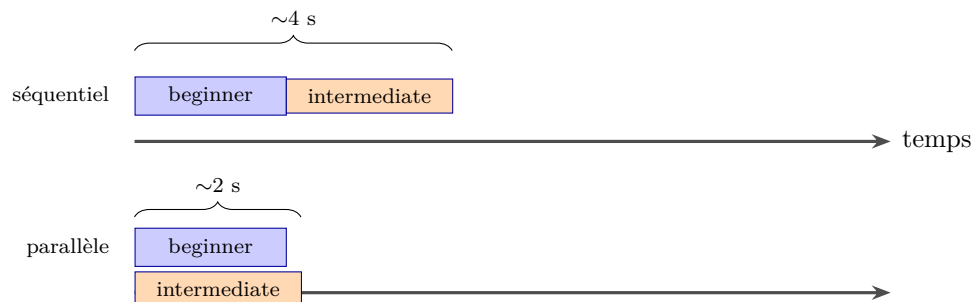
4.1.1 Architecture

Vue d'ensemble



`RunnableParallel` duplique l'input vers chaque branche, lance toutes les branches **en concurrence**, attend que toutes terminent, et retourne un **dict** dont les clés sont les noms de branches.

Diagramme temporel : séquentiel vs parallèle



Le gain est d'autant plus marqué qu'on a de branches : 3 chaînes en parallèle restent à la latence d'une seule chaîne (modulo le plafond de *rate limiting* du fournisseur).

4.1.2 Le code complet

```
1 from langchain_core.runnables import RunnableParallel
2
3 def explain_chain_for_level(level: str):
4     model = get_model().with_structured_output(Explanation)
5     return EXPLAIN_PROMPT.partial(level=level) | model
6
7
8 def build_multi_level_chain():
9     """Génère les versions débutant ET intermédiaire en un seul invoke."""
10    return RunnableParallel(
11        beginner=explain_chain_for_level("débutant"),
12        intermediate=explain_chain_for_level("intermédiaire"),
13    )
14
15
16 # Utilisation
17 multi = build_multi_level_chain()
18 results = multi.invoke({"concept": "les générateurs Python"})
```



```

19
20 # results est un dict.
21 # Les deux appels LLM ont été lancés en concurrence ; on n'a payé
22 # que la latence du plus lent.
23 print(results["beginner"].intuition)
24 print(results["intermediate"].intuition)

```

4.1.3 Explications pas à pas

Étape 1 — Le contrat de RunnableParallel

RunnableParallel prend un **ensemble de Runnable nommés**, généralement passés en kwargs :

```

1 RunnableParallel(
2     beginner=chain_a,
3     intermediate=chain_b,
4     advanced=chain_c,
5 )

```

À l'invocation :

1. L'input **x** est **dupliqué** vers chaque branche.
2. Toutes les branches s'exécutent en parallèle (via le pool de threads pour les chaînes synchrones, via l'évent loop asyncio pour les chaînes async).
3. LCEL attend que **toutes** terminent.
4. La sortie est un dict {**nom_branche**: **résultat**}.

Un seul input, plusieurs sorties. C'est le pattern « éventail » : on prend une question, on en demande plusieurs angles, et chaque branche travaille sur sa version.

Étape 2 — Le gain de latence n'est pas le gain de débit

Un appel LLM passe la quasi-totalité du temps à **attendre** la réponse du serveur. Sur le client, le thread est endormi. Lancer deux appels en parallèle ne consomme donc pas deux fois plus de CPU local : ça consomme deux *tickets* chez le fournisseur en même temps.

Mesure	Séquentiel	Parallèle (2 branches)
Latence totale	$T_1 + T_2$	$\max(T_1, T_2)$
Tokens consommés	$N_1 + N_2$	$N_1 + N_2$
Coût \$	idem	idem
CPU client	1 thread occupé	2 threads endormis sur I/O

*La parallélisation **ne réduit pas la facture** : tu paies toujours chaque token. Elle réduit uniquement la latence perçue par l'utilisateur. Pour un usage UI où le temps de réponse compte, c'est décisif. Pour un job batch nocturne, le gain est nul (et tu risques juste de saturer plus vite ton rate limit).*

Étape 3 — Le sucre syntaxique dict

RunnableParallel a deux formes équivalentes :

```

1 # Forme explicite
2 chain = RunnableParallel(beginner=a, intermediate=b)
3

```

```

4 # Forme dict (utile pour des noms non-identifieurs, ex. avec espaces)
5 chain = RunnableParallel({"beginner": a, "intermediate": b})

```

Et LCEL accepte même un **dict nu** dans une composition par `|` : il sera implicitement converti en `RunnableParallel`.

```

1 chain = some_prompt | {"answer": chain_a, "summary": chain_b}
2 # strictement équivalent à
3 chain = some_prompt | RunnableParallel(answer=chain_a, summary=chain_b)

```

Le **dict implicite** dans une chaîne LCEL signifie toujours `RunnableParallel`. Quand tu vois `{ "x": f, "y": g }` dans un `|`, lis « éventail vers `f` et `g`, sortie nommée ».

Étape 4 — `RunnableParallel` vs `RunnablePassthrough.assign`

Ces deux primitives se ressemblent, mais l'intention diffère :

<code>RunnableParallel</code>	<code>RunnablePassthrough.assign</code>
Renvoie un nouveau dict contenant uniquement les clés des branches.	Enrichit l'input existant : garde toutes les clés d'entrée <i>et</i> ajoute les nouvelles.
Sortie : <code>{nom1: ..., nom2: ...}</code> (les inputs disparaissent).	Sortie : <code>{**input, nom1: ..., nom2: ...}</code> (inputs préservés).
Usage typique : fan-out final (l'utilisateur veut <i>le résultat</i>).	Usage typique : ajout d'un champ calculé à mi-pipeline.

Si tu utilises `RunnableParallel` à la place de `.assign(...)` dans un pipeline, l'**input** d'origine **disparaît** dans la sortie. Les maillons suivants qui s'attendaient à lire la clé **concept** ne la trouveront plus. C'est l'un des bugs LCEL les plus fréquents.

Étape 5 — Limites et bonnes pratiques

- **Rate limits** : chaque branche compte. 5 branches en parallèle, c'est 5 appels simultanés chez le fournisseur — vérifie que tu ne dépasses pas le RPM (*requests per minute*) de ton compte.
- **Erreur partielle** : si une branche échoue, par défaut tout échoue. Pour tolérer les erreurs partielles, enveloppe chaque branche dans `.with_fallbacks([...])`.
- **Branches indépendantes uniquement** : si la branche B a besoin du résultat de la branche A, ce n'est pas du parallèle — c'est une séquence, à composer avec `|` et `.assign`.

4.1.4 Exécution et observation

Le code de l'exécuteur

```

1 import time
2
3 multi = build_multi_level_chain()
4
5 t0 = time.perf_counter()
6 results = multi.invoke({"concept": "les générateurs Python"})
7 elapsed = time.perf_counter() - t0
8
9 print(f"Durée RunnableParallel : {elapsed:.2f} s")
10
11 # Pour comparaison, version séquentielle

```

```

12 t0 = time.perf_counter()
13 r1 = explain_chain_for_level("débutant").invoke({"concept": "les générateurs Python"})
14 r2 = explain_chain_for_level("intermédiaire").invoke({"concept": "les générateurs
    ↪ Python"})
15 elapsed_seq = time.perf_counter() - t0
16
17 print(f"Durée séquentielle : {elapsed_seq:.2f} s")
18 print(f"Speedup : x{elapsed_seq / elapsed:.2f}")

```

Typiquement, on observe un speedup proche de $\times 1.8$ à $\times 2.0$ pour deux branches (le facteur 2 théorique étant légèrement entamé par l'overhead de *threading* et les variations de latence côté fournisseur).

4.2 Vérification dans LangSmith

L'arbre d'exécution attendu

```

RunnableParallel<beginner, intermediate>
  [beginner] RunnableSequence
    ChatPromptTemplate
    ChatAnthropic (structured_output) ← LLM #1
  [intermediate] RunnableSequence
    ChatPromptTemplate
    ChatAnthropic (structured_output) ← LLM #2

```

Les deux sous-séquences sont des **frères** sous le même `RunnableParallel`. LangSmith affiche un *timeline* où l'on voit clairement les deux barres se chevaucher temporellement.

Les quatre choses à regarder

1. **Timeline en haut du run racine** : deux barres parallèles. Si les barres sont en *série* (l'une après l'autre), c'est que `RunnableParallel` n'a pas été utilisé correctement (par exemple, tu as fait deux `invoke` séparés dans une boucle).
2. **Input des deux branches** : identique. C'est la duplication de l'input qu'on a évoquée.
3. **Output du run racine** : un dict avec les clés `beginner` et `intermediate`. Aucune autre clé.
4. **Onglet *Costs*** : les tokens et le \$ s'additionnent entre branches — la parallélisation ne change rien à la facture.

*Lancer deux appels LLM en parallèle dans LangChain, c'est littéralement **ajouter un mot** : `RunnableParallel`. Pas de `threads` à gérer, pas d'`asyncio.gather`, pas de `pool`. Le runtime LCEL fait tout, et le tracing LangSmith te montre la concurrence à l'œil nu.*

Récapitulatif

Ce qu'il faut retenir :

1. `RunnableParallel(nom1=chain1, nom2=chain2, ...)` duplique l'input vers chaque branche, exécute toutes les branches en **concurrence**, et renvoie un **dict** de résultats.
2. **Latence** : $\max(T_i)$ au lieu de $\sum T_i$. Coût en tokens **inchangé**. C'est un gain de latence, pas un gain de débit.
3. `prompt / {"a": chain_a, "b": chain_b}` est du sucre syntaxique : le dict est implicitement converti en `RunnableParallel`.

4. *Ne pas confondre avec `RunnablePassthrough.assign` : `Parallel` remplace l'input, `.assign` l'enrichit.*
5. *Dans `LangSmith`, la concurrence se voit immédiatement : deux barres de timeline qui se chevauchent sous le même nœud parent.*

5 Streaming token par token avec `.stream()`

Exercice 5. Streaming token par token

L'utilisateur pose une question au tuteur Python. Plutôt que d'attendre *toute* la réponse (souvent 2 à 5 secondes), on veut afficher la réponse **au fur et à mesure** qu'elle est générée, comme dans ChatGPT : le premier mot apparaît au bout de ~300 ms, puis les suivants s'écrivent en temps réel.

Contrainte : rester un `Runnable` LCEL classique, et afficher token par token dans le terminal, sans buffering perturbateur.

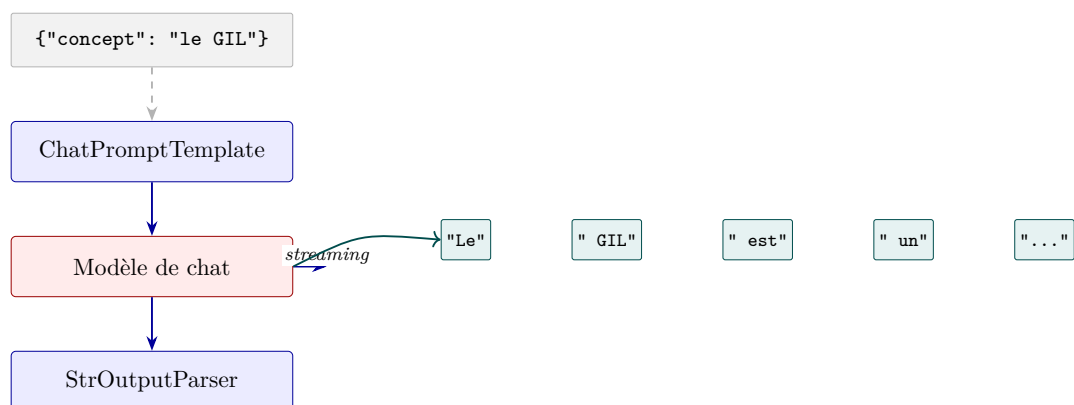
Objectifs pédagogiques

- Comprendre la différence entre `.invoke()` (réponse unique) et `.stream()` (itérateur de morceaux).
- Savoir construire une chaîne *streamable* et l'afficher au fil de l'eau dans un terminal ou une UI.
- Comprendre pourquoi `with_structured_output` ne stream pas bien et quand basculer sur `StrOutputParser`.
- Connaître les pièges classiques du streaming (buffering, flush, retour chariot).

5.1 Solution

5.1.1 Architecture

Vue d'ensemble



pour `chunk` in `chain.stream(...)` : `print(chunk, end="", flush=True)`

L'API du modèle ouvre une connexion *server-sent events* avec le fournisseur. Chaque token reçu est immédiatement émis par la chaîne. Tant qu'on consomme l'itérateur, les tokens arrivent.

5.1.2 Le code complet

```
1 from langchain_core.prompts import ChatPromptTemplate
2 from langchain_core.output_parsers import StrOutputParser
3 from config import get_model
4
5 STREAM_PROMPT = ChatPromptTemplate.from_messages([
6     ("system", "Tu es un assistant pédagogique Python concis et précis."),
7     ("human", "Explique en 2 paragraphes : {concept}"),
8 ])
9
10
11 def build_streaming_chain():
12     """Chaîne texte simple, idéale pour du streaming dans un terminal/UI."""
13     return STREAM_PROMPT | get_model() | StrOutputParser()
14
15
16 # Utilisation
17 streamer = build_streaming_chain()
18 for chunk in streamer.stream({"concept": "le GIL en Python"}):
19     print(chunk, end="", flush=True)
20 print() # saut de ligne final
```

5.1.3 Explications pas à pas

Étape 1 — .invoke() vs .stream()

.invoke(input)	.stream(input)
Renvoie une valeur unique : la réponse complète.	Renvoie un <i>itérateur</i> de morceaux (chunks).
Le code attend que tout soit prêt avant de continuer.	Le code itère et reçoit les morceaux dès qu'ils arrivent.
Latence perçue = latence totale.	Latence perçue = latence du <i>premier</i> token (300 ms).
Bien pour batch, scripts, données structurées.	Bien pour UI temps réel, terminal interactif.

Le *time-to-first-token (TTFT)* est ce qui rend une interface « vivante ». Même si la durée totale est identique, voir la réponse **commencer** à apparaître au bout de 300 ms change radicalement la sensation d'attente.

Étape 2 — Pourquoi StrOutputParser ?

Par défaut, un modèle de chat renvoie des AIMessageChunk (des fragments d'AIMessage). En streaming, on ne veut pas avoir à faire `chunk.content` dans la boucle d'affichage — on veut directement le *texte*. C'est exactement le rôle de `StrOutputParser` :

Entrée	Sortie
AIMessageChunk(content="Le")	"Le"
AIMessageChunk(content=" GIL")	" GIL"
...	...

```
1 # Sans StrOutputParser
2 for chunk in (STREAM_PROMPT | get_model()).stream({"concept": "X"}):
3     print(chunk.content, end="", flush=True) # accès manuel à .content
```

```

4
5 # Avec StrOutputParser
6 for chunk in (STREAM_PROMPT | get_model() | StrOutputParser()).stream({"concept": "X"}):
7     print(chunk, end="", flush=True) # le chunk EST une str

```

Étape 3 — Pourquoi with_structured_output stream mal



Un schéma Pydantic est **tout ou rien** : tant que tous les champs requis ne sont pas remplis, l'objet ne peut pas être validé. Le modèle peut bien streamer le JSON token par token, on ne peut pas *désérialiser* en cours de route — la moitié des champs manquent.

Appeler `.stream()` sur une chaîne `with_structured_output` ne plantera pas, mais souvent la chaîne accumule tout en interne et n'émet qu'un seul gros chunk à la fin. Le bénéfice du streaming disparaît. Pour un streaming réel, soit on accepte du texte (`StrOutputParser`), soit on utilise des techniques avancées (parsing JSON incrémental, mais c'est une autre histoire).

Étape 4 — La boucle d'affichage côté Python

```

1 for chunk in streamer.stream({"concept": "le GIL en Python"}):
2     print(chunk, end="", flush=True)
3 print()

```

Trois détails subtils mais importants :

- `end=""` : empêche `print` d'ajouter un `\n` après chaque chunk. Sans ça, chaque token est affiché sur sa propre ligne — illisible.
- `flush=True` : force l'écriture sur le terminal **immédiatement**. Python bufferise stdout par défaut (par bloc de 4 ko ou par ligne) ; sans flush, on voit la réponse arriver par paquets, pas par tokens.
- Le `print()` final est juste là pour le saut de ligne après toute la réponse.

`flush=True` est le secret du streaming visible. Si tu oublies, tu as l'impression que ton streaming ne marche pas — alors qu'il marche, mais c'est le terminal qui retient les caractères. C'est la question #1 posée en debug.

Étape 5 — Cas async : `.astream()`

Pour une UI web ou un endpoint FastAPI, on veut la version asynchrone :

```

1 async def stream_to_websocket(websocket, concept: str):
2     async for chunk in streamer.astream({"concept": concept}):
3         await websocket.send_text(chunk)

```

La sémantique est identique à `.stream()`, mais on utilise `async for` et chaque `await` libère l'événement pour les autres connexions. C'est le pattern à connaître pour servir N clients simultanément.

5.2 Vérification dans LangSmith

L'arbre d'exécution attendu

```
RunnableSequence
  ChatPromptTemplate
    ChatAnthropic (streaming=True)
      Tokens : "Le", " GIL", " est", " un", ...
        StrOutputParser
```

LangSmith reconstitue la réponse complète à la fin et l'affiche dans l'onglet *Output*, mais conserve la durée *end-to-end* et le *TTFT* dans les métriques.

Les trois choses à regarder

1. **Métrique *Time to first token*** : temps entre l'appel et le premier token reçu. C'est la métrique qui détermine la *sensation de vivacité* de ton UI. Souvent affichée sous `ttft_ms` dans les détails du run.
2. **Onglet *Output* du nœud `ChatAnthropic`** : l'`AIMessage` reconstitué (LangSmith concatène les chunks pour l'affichage). Tu ne vois *pas* chaque token séparément dans l'UI, mais la trace HTTP sous-jacente est bien du streaming.
3. **Type du modèle dans la trace** : `streaming=True` ou un flag équivalent. Si tu vois `streaming=False`, ton appel est en mode bloquant.

Compare deux runs côte à côte : l'un avec `.invoke`, l'autre avec `.stream`, sur le même prompt. Le total de tokens et le coût \$ sont identiques. Mais le run streamé a un TTFT $\sim 10\times$ plus court. C'est, mesurée, la valeur ajoutée du streaming.

Récapitulatif

Ce qu'il faut retenir :

1. `chain.stream(input)` renvoie un **itérateur** de morceaux ; `chain.invoke(input)` renvoie la réponse complète. Même chaîne, deux modes de consommation.
2. Pour streamer du texte, terminer la chaîne par `StrOutputParser` pour que chaque morceau soit déjà une `str` (et pas un `AIMessageChunk`).
3. `with_structured_output` ne streame pas vraiment : un schéma est tout-ou-rien, on ne peut pas désérialiser un objet partiel. Pour du streaming réel, accepter du texte.
4. Dans la boucle d'affichage : `print(chunk, end="", flush=True)`. Sans `flush=True`, les buffers du terminal masquent l'effet streaming.
5. Pour le web ou les apps `async`, utiliser `.astream()` avec `async for`. Sémantique identique, mais l'event loop reste libre pour les autres connexions.

Deuxième partie

Pour aller plus loin

6 Multi-niveaux 3× : parallèle vs séquentiel

Exercice bonus 1. Mesurer le speedup d'un RunnableParallel à 3 branches

Ajoute un niveau "avancé" à la chaîne multi-niveaux et lance les **3 niveaux en parallèle**. Mesure le temps d'exécution **séquentiel** (3 `.invoke()` successifs) et le temps **parallèle** (un seul `.invoke()` sur un `RunnableParallel` à 3 clés). Compare et calcule le *speedup*.

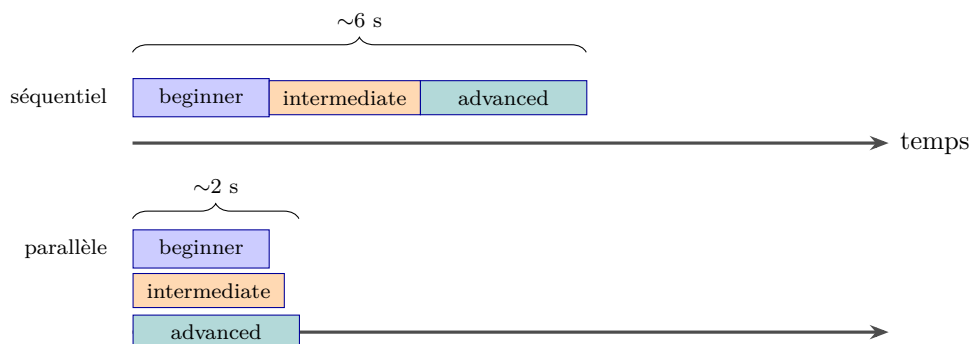
Contrainte : utiliser `time.perf_counter()` pour les mesures (plus précis que `time.time()`). Afficher le rapport `seq_time / par_time`.

Objectifs pédagogiques

- Vérifier expérimentalement que `RunnableParallel` donne une latence en $\max(T_i)$ et non $\sum T_i$.
- Observer la dégradation du speedup quand le nombre de branches augmente (rate limiting côté fournisseur).
- Savoir mesurer proprement la latence avec `time.perf_counter()`.

6.1 Solution

6.1.1 Architecture



Trois branches en parallèle restent à la latence d'une seule chaîne. Le speedup attendu est de $\times 3$ en théorie, $\times 2.5$ à $\times 2.8$ en pratique (overhead de *threading*, rate limits, variations côté fournisseur).

6.1.2 Le code complet

```
1 import time
2 from langchain_core.runnables import RunnableParallel
3 from phase1_explain import explain_chain_for_level
4
5
6 def exercise_1() -> None:
7     concept = "les décorateurs Python"
8
9     # --- Construction des 3 chaînes spécialisées ---
10    beginner      = explain_chain_for_level("débutant")
11    intermediate  = explain_chain_for_level("intermédiaire")
12    advanced      = explain_chain_for_level("avancé")
```



```

13
14 # --- Version SÉQUENTIELLE : 3 invokes successifs ---
15 t0 = time.perf_counter()
16 seq_results = {
17     "beginner":    beginner.invoke({"concept": concept}),
18     "intermediate": intermediate.invoke({"concept": concept}),
19     "advanced":    advanced.invoke({"concept": concept}),
20 }
21 seq_time = time.perf_counter() - t0
22 print(f"Séquentiel : {seq_time:.2f}s pour 3 niveaux")
23
24 # --- Version PARALLÈLE : un seul invoke sur un RunnableParallel ---
25 parallel = RunnableParallel(
26     beginner=beginner,
27     intermediate=intermediate,
28     advanced=advanced,
29 )
30 t0 = time.perf_counter()
31 par_results = parallel.invoke({"concept": concept})
32 par_time = time.perf_counter() - t0
33 print(f"Parallèle : {par_time:.2f}s pour 3 niveaux")
34 print(f"→ Speedup : ×{seq_time / par_time:.2f}")

```

6.1.3 Explications pas à pas

Étape 1 — Construire les trois chaînes spécialisées

`explain_chain_for_level` est la fabrique qu'on a vue dans le concept 3 de la Phase 1. Trois appels suffisent à produire trois `Runnable`, chacun avec son niveau figé via `.partial()`. Pas de duplication de prompt — c'est tout l'intérêt du pattern fabrique.

Étape 2 — La mesure séquentielle

Trois `.invoke()` successifs s'exécutent en **série** : le thread Python attend la réponse du serveur entre chaque appel. Avec un modèle moyen, chaque `.invoke()` prend ~ 2 s ; total ~ 6 s.

Étape 3 — La mesure parallèle

`RunnableParallel(beginner=..., intermediate=..., advanced=...)` crée un seul `Runnable` qui, à l'invocation, duplique l'input vers les 3 branches et les exécute en **concurrency** (via un pool de threads). On attend que les trois terminent et on récupère un dict `{"beginner": ..., "intermediate": ..., "advanced": ...}`.

Étape 4 — Le speedup observé

En pratique, on observe :

- **Séquentiel** : ~ 5 à 7 s
- **Parallèle** : ~ 2 à 3 s
- **Speedup** : $\times 2.0$ à $\times 2.8$

Le facteur 3 théorique n'est jamais atteint à cause de l'overhead de threading, des variations de latence côté serveur, et — surtout — des *rate limits* du fournisseur quand les 3 appels arrivent en même temps.

Coût en tokens : strictement identique entre séquentiel et parallèle. La parallélisation ne change rien à la facture, seulement à la latence perçue.

Sortie de l'exécution

```
=====
EXERCICE 1 - Parallèle vs séquentiel sur 3 niveaux
=====
```

Séquentiel : 22.86s pour 3 niveaux

Parallèle : 8.98s pour 3 niveaux

→ Speedup : $\times 2.55$

Extrait niveau avancé :

Un décorateur est une fonction qui prend une autre fonction en entrée, la modifie ou l'enveloppe avec du code supplémentaire, puis retourne la version modifiée. C'est comme un emballage cadeau qui ajo...

6.2 Vérification dans LangSmith

L'arbre d'exécution affiche un seul `RunnableParallel` parent avec **3 frères** qui se chevauchent temporellement dans la timeline en haut du run :

```
RunnableParallel<beginner, intermediate, advanced>
    [beginner] RunnableSequence ← LLM #1
    [intermediate] RunnableSequence ← LLM #2
    [advanced] RunnableSequence ← LLM #3
```

Dans la *timeline*, les 3 barres démarrent au même instant (duplication de l'input) et finissent à $\max(T_i)$. Si l'une d'elles prend visiblement plus de temps (variations de tokens, retry interne), le total dépend d'elle.

Récapitulatif

Ce qu'il faut retenir :

1. `RunnableParallel(a=..., b=..., c=...)` parallélise N chaînes sur le même input. La latence reste à $\max(T_i)$.
2. `time.perf_counter()` est l'outil standard pour mesurer une durée en Python (monotone, haute résolution).
3. Le speedup réel plafonne à $\sim 80\%$ du facteur théorique à cause des rate limits du fournisseur et de l'overhead de threading.
4. Le coût en tokens reste identique — paralléliser n'est qu'un gain de latence, jamais de débit ni d'argent.

7 .batch() vs boucle for

Exercice bonus 2. Paralléliser plusieurs *inputs* sur la même chaîne

Utilise `.batch()` au lieu de `.invoke()` pour expliquer une **liste de concepts** en parallèle. Compare avec une boucle `for` qui appelle `.invoke()` successivement. Calcule le speedup observé.

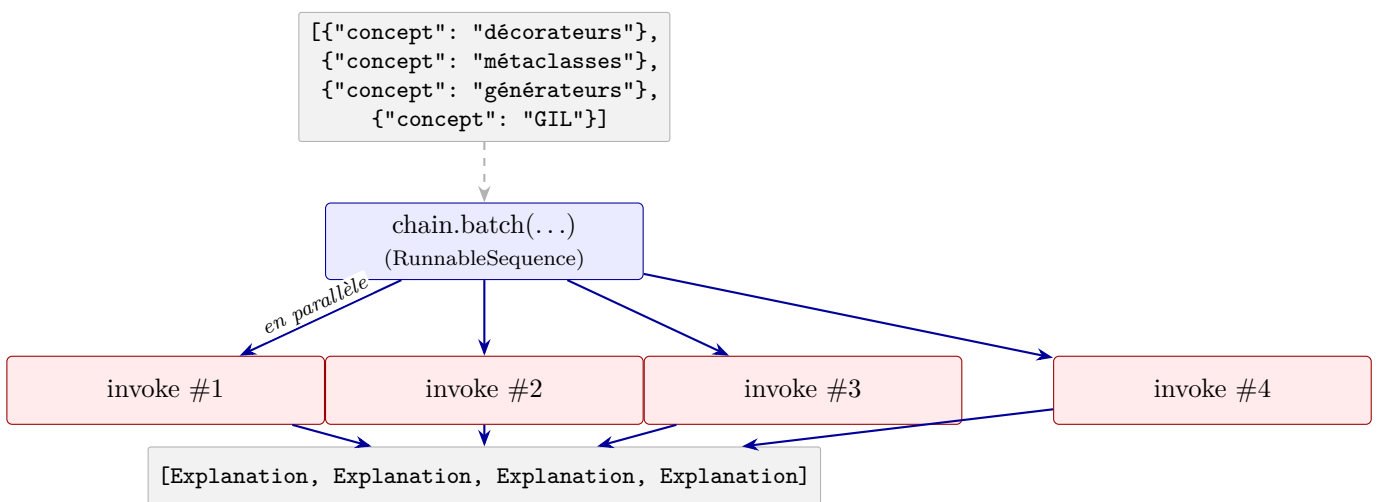
Contrainte : utiliser une chaîne intermédiaire et une liste d'au moins 4 concepts variés (`["décorateurs", "métaclasses", "générateurs", "GIL"]` par exemple).

Objectifs pédagogiques

- Distinguer `.batch()` (plusieurs inputs, *même* chaîne) de `RunnableParallel` (même input, plusieurs chaînes).
- Savoir invoquer une chaîne `.batch(inputs)` avec une liste de dicts.
- Contrôler le degré de parallélisme via `config={"max_concurrency": N}` pour respecter les rate limits.

7.1 Solution

7.1.1 Architecture



Une seule chaîne, N invocations parallèles. Symétrique mais distinct de `RunnableParallel` qui prend N chaînes différentes sur le même input.

7.1.2 Le code complet

```
1 import time
2 from phase1_explain import explain_chain_for_level
3
4
5 def exercise_2() -> None:
6     concepts = ["les décorateurs", "les métaclasses", "les générateurs", "le GIL"]
7     chain = explain_chain_for_level("intermédiaire")
8
9     # --- Version BOUCLE : un invoke après l'autre ---
10    t0 = time.perf_counter()
11    loop_results = [chain.invoke({"concept": c}) for c in concepts]
12    loop_time = time.perf_counter() - t0
13    print(f"Boucle for : {loop_time:.2f}s pour {len(concepts)} concepts")
```

```

14
15 # --- Version BATCH : un seul appel, parallélisé sous le capot ---
16 inputs = [{"concept": c} for c in concepts]
17 t0 = time.perf_counter()
18 batch_results = chain.batch(inputs)
19 batch_time = time.perf_counter() - t0
20 print(f"batch()      : {batch_time:.2f}s pour {len(concepts)} concepts")
21 print(f"→ Speedup   : ×{loop_time / batch_time:.2f}")
22
23 # Contrôler le degré de parallélisme pour éviter de saturer les rate limits :
24 # chain.batch(inputs, config={"max_concurrency": 5})

```

7.1.3 Explications pas à pas

Étape 1 — Le contrat de `.batch()`

`chain.batch(inputs)` accepte une **liste** de dicts et renvoie une liste de résultats **dans le même ordre** que l'entrée :

```

1 inputs = [
2     {"concept": "décorateurs"},      # → results[0]
3     {"concept": "métaclasses"},     # → results[1]
4     {"concept": "générateurs"},     # → results[2]
5 ]
6 results = chain.batch(inputs)      # même ordre garanti

```

Sous le capot, LCEL lance les N invocations en parallèle (threads ou asyncio selon le contexte) et collecte les résultats.

Étape 2 — `.batch()` vs `RunnableParallel`

<code>chain.batch(inputs)</code>	<code>RunnableParallel(a, b, c)</code>
Une seule chaîne, plusieurs inputs.	N chaînes différentes, un seul input.
Entrée : <code>list[dict]</code> (N inputs).	Entrée : <code>dict</code> unique (duplicué).
Sortie : <code>list[result]</code> (même ordre).	Sortie : <code>dict[branch_name, result]</code> .
Usage : explorer N requêtes <i>indépendantes</i> .	Usage : N angles d'analyse d'une requête.

Règle mnémotechnique : *plusieurs inputs, même chaîne* → `.batch()` ; *même input, plusieurs chaînes* → `RunnableParallel`.

Étape 3 — Contrôler le parallélisme avec `max_concurrency`

Par défaut, `.batch(N inputs)` lance les N appels simultanément. Si N est grand (50, 100...), le fournisseur va te *rate limiter*. Le contrôle se fait via `config` :

```

1 # Limiter à 5 appels concurrents
2 results = chain.batch(inputs, config={"max_concurrency": 5})

```

LCEL exécute alors les inputs en **vagues de 5** : 5 partent en parallèle, on attend qu'ils finissent, puis les 5 suivants. Pour 50 inputs avec `max_concurrency=5`, on a 10 vagues séquentielles.

Étape 4 — Le speedup observé

Pour 4 inputs à ~ 2 s chacun :

- **Boucle for** : ~ 8 s
- **batch()** : ~ 2 à 3 s
- **Speedup** : $\times 3$ à $\times 4$

Sortie de l'exécution

```
=====
EXERCICE 2 - batch() vs boucle for
=====
Boucle for : 32.30s pour 4 concepts
batch() : 8.30s pour 4 concepts
→ Speedup :  $\times 3.89$ 
Exemple (les décorateurs) :
Un décorateur est une fonction qui « enveloppe » une autre fonction pour modifier son comportement sans
changer son code original. C'est comme ajouter une couche de papier cadeau autour d'un cadeau : le...
```

7.2 Vérification dans LangSmith

Le run racine s'appelle `RunnableSequence.batch` et contient N sous-runs frères, chacun étant un `RunnableSequence` (la chaîne exécutée sur un input). Dans la timeline, on voit clairement les N barres parallèles. Le rapport `total_tokens` sur le run racine agrège les N appels.

Dans LangSmith, un `.batch()` produit un seul run parent avec N enfants frères. Si tu vois N runs racines distincts (au lieu d'un parent avec N enfants), c'est que tu as utilisé une boucle `for` et pas `.batch()`.

Récapitulatif

Ce qu'il faut retenir :

1. `chain.batch(inputs)` parallélise N invocations d'une **même** chaîne sur une liste de N dicts d'entrée.
2. Ne pas confondre avec `RunnableParallel` : `batch` est « plusieurs inputs / même chaîne », `RunnableParallel` est « même input / plusieurs chaînes ».
3. L'ordre de sortie est garanti identique à l'ordre d'entrée.
4. `config={"max_concurrency": N}` limite le nombre d'appels concurrents pour respecter les `rate limits`.
5. *Speedup typique* : $\times 3$ à $\times 4$ pour 4 inputs, avec plafonnement progressif quand N augmente.

8 Schéma Pydantic enrichi avec un champ analogy

Exercice bonus 3. Étendre un schéma Pydantic sans toucher au prompt

Ajoute un champ `analogy: str` au schéma Pydantic de la Phase 1 (analogie avec un objet ou une situation du quotidien). Vérifie que le modèle remplit ce champ **sans aucune modification du prompt**.

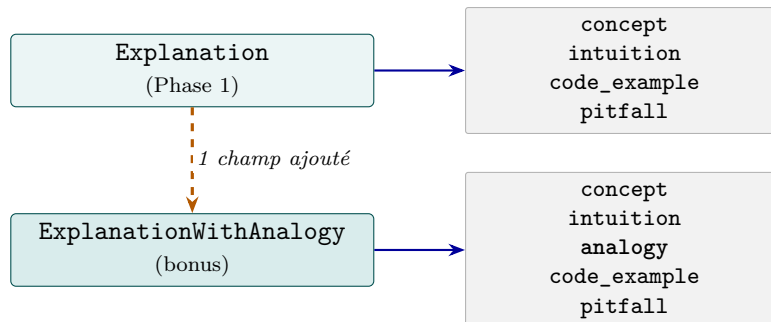
Contrainte : la `description=` du nouveau champ doit suffire à piloter le contenu. Pas de `ChatPromptTemplate` modifié, pas de système re-rédigé. Juste le schéma.

Objectifs pédagogiques

- Mesurer concrètement la *viscosité* (faible) d'un changement avec `with_structured_output`.
- Comprendre que `Field(description=...)` est un véritable levier de *prompt engineering*.
- Voir la mise à jour du tool schema dans LangSmith en direct.

8.1 Solution

8.1.1 Architecture



Une seule ligne change : l'ajout du champ `analogy: str = Field(...)` dans la classe. Le reste du code applicatif est strictement identique.

8.1.2 Le code complet

```
1 from pydantic import BaseModel, Field
2 from phase1_explain import EXPLAIN_PROMPT
3 from config import get_model
4
5
6 class ExplanationWithAnalogy(BaseModel):
7     """Version enrichie : on rajoute une analogie du quotidien."""
8
9     concept: str = Field(description="Le concept expliqué.")
10    intuition: str = Field(description="Explication intuitive en 3 phrases.")
11    analogy: str = Field(
12        description=(
13            "Une analogie avec un objet ou une situation du quotidien "
14            "(cuisine, sport, bureau...) qui éclaire le concept pour un débutant. "
15            "Une seule phrase percutante."
16        )
17    )
18    code_example: str = Field(description="Un exemple de code minimal et commenté.")
19    pitfall: str = Field(description="Un piège classique sur ce concept.")
20
```

```

21
22 def exercise_3() -> None:
23     # Une seule ligne change : le schéma passé à with_structured_output.
24     model = get_model().with_structured_output(ExplanationWithAnalogy)
25     chain = EXPLAIN_PROMPT.partial(level="débutant") | model
26
27     for concept in ["les décorateurs", "le polymorphisme", "le contexte avec `with`"]:
28         result = chain.invoke({"concept": concept})
29         print(f"\n--- {concept} ---")
30         print(f"Analogie : {result.analogy}")

```

8.1.3 Explications pas à pas

Étape 1 — Étendre la classe Pydantic

L'ajout d'un champ se fait comme dans n'importe quel `BaseModel` Pydantic : une nouvelle ligne avec un nom, un type, et un `Field(description=...)`. Le champ devient automatiquement **requis** (Pydantic exigera sa présence).

Étape 2 — La seule ligne d'application qui change

```

1 # Avant
2 model = get_model().with_structured_output(Explanation)
3
4 # Après
5 model = get_model().with_structured_output(ExplanationWithAnalogy)

```

C'est tout. Le prompt système n'a pas bougé. Le `.partial()` n'a pas bougé. Le `|` n'a pas bougé. Le LLM va recevoir le nouveau tool schema et le remplir intégralement.

Étape 3 — La `description=` fait tout le travail

Le LLM ne sait pas *a priori* ce qu'on attend d'un champ `analogy`. C'est la `description=` qui le lui dit :

« Une analogie avec un objet ou une situation du quotidien (cuisine, sport, bureau...) qui éclaire le concept pour un débutant. Une seule phrase percutante. »

Trois éléments importants dans cette description :

- **Domaine** : « cuisine, sport, bureau » — donne des exemples concrets de registre attendu.
- **Public** : « pour un débutant » — calibre la simplicité.
- **Forme** : « une seule phrase percutante » — limite la longueur, force la concision.

C'est du *prompt engineering* appliqué directement au schéma.

Étape 4 — La force de `with_structured_output`

C'est cette élasticité qu'on cherche en production : les contrats de sortie évoluent (un manager veut un nouveau champ, un produit veut une nouvelle métadonnée, etc.). Avec un parseur texte classique, il faudrait rééditer le prompt, gérer les cas d'oubli, écrire du code de fallback. Avec `with_structured_output`, on ajoute le champ et **c'est tout**.

***Coût** : la description du nouveau champ ajoute quelques tokens au prompt (le schéma est envoyé à chaque appel). Le coût marginal d'un champ supplémentaire est de ~30 à 80 tokens — négligeable.*

Sortie de l'exécution

```
=====
EXERCICE 3 - Schéma Pydantic enrichi avec un champ analogy
=====
```

— les décorateurs —

Analogie : C'est comme emballer un cadeau : tu prends ta fonction (le cadeau), tu la mets dans du papier cadeau (le décorateur), et maintenant le cadeau a une belle présentation en plus, sans que le contenu ne change.

— le polymorphisme —

Analogie : C'est comme une télécommande universelle : le bouton « play » fait jouer la musique sur un haut-parleur, mais lance la vidéo sur un téléviseur. Même bouton, résultats différents selon l'appareil.

— le contexte avec with —

Analogie : C'est comme emprunter un livre à la bibliothèque : **with** c'est le système qui te force à le rendre à la date limite, même si tu l'oublies. Sans **with**, tu dois te souvenir tout seul de le rendre.

8.2 Vérification dans LangSmith

Dans l'onglet *Tool calls* du **ChatAnthropic**, on voit le nouveau tool schema avec 5 champs au lieu de 4. Le champ **analogy** apparaît avec sa description complète. L'output du run racine est un objet **ExplanationWithAnalogy** dont les 5 champs sont remplis.

```
ChatAnthropic (structured_output=ExplanationWithAnalogy)

Tools : [{
    name: "ExplanationWithAnalogy",
    parameters: { concept, intuition,
                  analogy, code_example, pitfall }
}]
```

*Modifier la **description**= d'un champ et relancer : tu vois immédiatement le changement dans le tool schema envoyé au LLM, et la sortie générée se met à jour conformément. C'est la boucle la plus rapide de prompt engineering qu'on puisse avoir.*

Récapitulatif

Ce qu'il faut retenir :

1. Ajouter un champ à un schéma Pydantic = **une ligne**, plus **un changement** du nom passé à **with_structured_output**.
2. La **description**= d'un **Field** est une partie du prompt. Soigne-la (registre, public, forme).
3. Le coût marginal d'un champ ajouté est négligeable (~30 à 80 tokens supplémentaires).
4. Dans LangSmith, le tool schema reflète immédiatement les changements de description — boucle de prompt engineering ultra-rapide.

9 Router avec RunnableBranch et classifieur LLM

Exercice bonus 4. Aiguillage dynamique vers 3 sous-chaînes spécialisées

Crée un *router* qui choisit automatiquement entre 3 sous-chaînes selon que le concept Python est de catégorie **syntaxe** (def, lambda, comprehensions...), **paradigme** (POO, fonctionnel, polymorphisme...) ou **écosystème** (pip, venv, pytest...). Utilise une mini-chaîne LLM de classification en amont pour décider du routage.

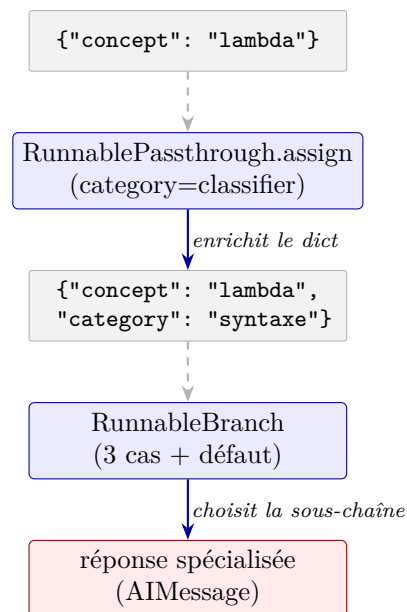
Contrainte : tout doit rester un seul Runnable LCEL pour préserver streaming, batching, et tracing LangSmith unifié.

Objectifs pédagogiques

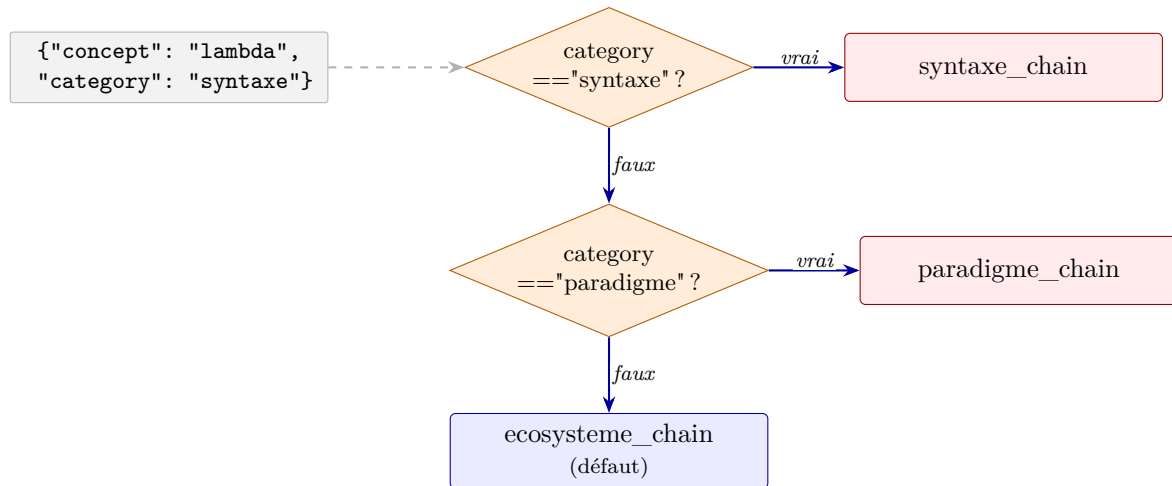
- Maîtriser le **pattern router** : classifieur LLM + sous-chaînes spécialisées + RunnableBranch.
- Utiliser `Literal[...]` dans un schéma Pydantic pour garantir un ensemble fermé de catégories.
- Combiner `RunnablePassthrough.assign` et `RunnableBranch` pour enrichir puis brancher.
- Repérer dans LangSmith les **trois appels LLM** qu'un router engendre par requête (et le surcoût associé).

9.1 Solution

9.1.1 Architecture



Zoom sur le RunnableBranch



9.1.2 Le code complet

```
1 from typing import Literal
2 from pydantic import BaseModel, Field
3 from langchain_core.prompts import ChatPromptTemplate
4 from langchain_core.runnables import (
5     RunnableBranch, RunnableLambda, RunnablePassthrough,
6 )
7 from config import get_model
8
9
10 class CategoryResult(BaseModel):
11     """Catégorie d'un concept Python pour router vers la bonne sous-chaîne."""
12     category: Literal["syntaxe", "paradigme", "ecosysteme"] = Field(
13         description=(
14             "Catégorise le concept :\n"
15             " - 'syntaxe' : éléments du langage (def, lambda, comprehensions, with, \n"
16             "   ↳ yield...)\n"
17             " - 'paradigme' : concepts de design (POO, fonctionnel, héritage, \n"
18             "   ↳ polymorphisme...)\n"
19             " - 'ecosysteme' : outils/bibliothèques (pip, venv, pytest, numpy, \n"
20             "   ↳ requests...)"
21         )
22     )
23
24 def build_classifier():
25     """Mini-chaîne qui catégorise un concept en 3 classes."""
26     classify_prompt = ChatPromptTemplate.from_messages([
27         ("system", "Tu classes des concepts Python en 3 catégories. "
28             "Ne réponds que par la catégorie."),
29         ("human", "Concept à catégoriser : {concept}"),
30     ])
31     return (
32         classify_prompt
33         | get_model().with_structured_output(CategoryResult)
34         | RunnableLambda(lambda r: r.category)
35     )
36
37 def build_specialized_chain(focus: str):
```

```

37     """Construit une sous-chaîne avec un prompt système orienté `focus`."""
38     prompt = ChatPromptTemplate.from_messages([
39         ("system",
40          "Tu es un assistant Python. Le concept à expliquer relève de la "
41          f"catégorie '{focus}'. Adapte ton explication en conséquence :\n" +
42          {
43              "syntaxe": "insiste sur la syntaxe exacte, montre un mini exemple et un
44                        ↪ anti-exemple.",
45              "paradigme": "insiste sur la philosophie, le 'pourquoi', les implications
46                        ↪ de design.",
47              "ecosysteme": "insiste sur l'usage pratique, l'installation, les commandes
48                        ↪ shell typiques.",
49          }[focus]),
50         ("human", "Explique : {concept}"),
51     ])
52     return prompt | get_model()
53
54 def build_router():
55     """Le routeur complet : classifie puis branche."""
56     classifieur = build_classifieur()
57     syntaxe_chain = build_specialized_chain("syntaxe")
58     paradigme_chain = build_specialized_chain("paradigme")
59     ecosysteme_chain = build_specialized_chain("ecosysteme")
60
61     branch = RunnableBranch(
62         (lambda x: x["category"] == "syntaxe", syntaxe_chain),
63         (lambda x: x["category"] == "paradigme", paradigme_chain),
64         ecosysteme_chain, # défaut : si rien ne matche
65     )
66
67     return RunnablePassthrough.assign(category=classifieur) | branch

```

9.1.3 Explications pas à pas

Étape 1 — Le vocabulaire de routage (CategoryResult)

Avant de router, il faut un **ensemble fermé** de catégories. Le `Literal["syntaxe", "paradigme", "ecosysteme"]` est crucial : il *garantit* que le LLM ne pourra pas renvoyer "truc" ou "je sais pas". La `description` du champ sert de mini-prompt qui explique au modèle *comment* classer.

Étape 2 — Le classifieur

Trois maillons en pipeline :

1. `classify_prompt : {"concept": "lambda"} → messages formatés.`
2. `model.with_structured_output(CategoryResult) : messages → objet CategoryResult(category="syntaxe")`
3. `RunnableLambda(lambda r: r.category) : objet → chaîne "syntaxe".`

Le maillon 3 n'est pas cosmétique : il rend la sortie utilisable comme **valeur de comparaison** dans les prédicats du `RunnableBranch`.

Étape 3 — Les sous-chaînes spécialisées

`build_specialized_chain(focus)` est une *fabrique* : on l'appelle 3 fois avec des `focus` différents et on obtient 3 `Runnable` distincts, chacun avec sa personnalité dans le `system prompt`.

*La spécialisation se fait à la **construction**, pas à l'appel. `build_specialized_chain("syntaxe")` renvoie un `Runnable` prêt à l'emploi qui ne sait répondre que dans le style « *syntaxe* ».*

Étape 4 — Le RunnableBranch

RunnableBranch est le if/elif/else de LCEL. Trois points clés :

- Les prédicats sont évalués **dans l'ordre**, le premier qui renvoie **True** gagne (court-circuit).
- Si *aucun* prédicat ne matche, la chaîne par défaut s'exécute — elle est **obligatoire**.
- La sous-chaîne sélectionnée reçoit **l'input complet** (le dict entier), pas juste le résultat du prédicat.

Étape 5 — L'enrichissement avant branchement

Le RunnableBranch a besoin que `category` soit **déjà dans l'input** pour que ses prédicats puissent le lire. C'est le rôle de `RunnablePassthrough.assign` :

RunnablePassthrough.assign(category=classifier) appelle le classifieur et ajoute le résultat sous la clé `category`, sans toucher aux clés existantes. C'est l'équivalent d'un `SELECT *, expr AS col` en SQL.

Sortie de l'exécution

```
=====
EXERCICE 4 - Router avec RunnableBranch
=====
[ syntaxe] les list comprehensions
→ # List Comprehensions en Python  ## Concept  Une list comprehension est une syntaxe compacte
pour créer une liste en appliquant une expression à chaque élément d'une séquence, avec possibilité de f...
[ paradigme] le polymorphisme
→ # Le Polymorphisme : Une Philosophie de Flexibilité  ## L'Essence Philosophique  Le polymorphisme
incarne une idée fondamentale : « une même interface, plusieurs formes ». C'est la capacité d'une en...
[ecosysteme] pytest
→ # pytest – Framework de Test Python  ## Concept Clé  pytest est un framework de test moderne et
puissant pour Python. Il simplifie l'écriture et l'exécution de tests avec une syntaxe minimaliste...
[ syntaxe] lambda
→ # Lambda en Python  ## Concept  Une lambda est une fonction anonyme (sans nom) définie en une
seule ligne. Elle permet de créer rapidement des petites fonctions sans utiliser def.  ## Syntaxe exa...
```

9.2 Vérification dans LangSmith

L'arbre d'exécution

Pour le concept "lambda", on observe **3 appels LLM** :

```
RunnableSequence (router)
    RunnableAssign<category>
        classifier ← LLM #1 (structured_output=CategoryResult)
            → "syntaxe"
        RunnableBranch
            branch[0]: syntaxe_chain
            ChatAnthropic ← LLM #2 (réponse spécialisée)
```

Les quatre choses à regarder

1. **L'arbre lui-même** : tu vois *visuellement* le passage par le classifieur puis la branche choisie. C'est l'aiguillage dynamique en image.
2. **Premier ChatAnthropic** (classifieur) : onglet *Output* → l'objet `CategoryResult` et le tool-call que Claude a utilisé.

3. **Deuxième ChatAnthropic** (réponse spécialisée) : le system prompt commence par « Tu es un assistant Python...catégorie 'syntaxe' ». Précieux pour debugger.
4. **Onglet Costs** : tokens et coût agrégés sur les 2 (ou 3, si on garde le `debug_chain`) appels. Permet de chiffrer le surcoût du pattern router.

*Compare deux runs dans l'UI : "lambda" déclenche `syntaxe_chain`, "pytest" déclenche `ecosysteme_chain`. On voit littéralement la **branche choisie** changer selon le concept.*

Récapitulatif

Ce qu'il faut retenir :

1. **Pattern router** = classifieur LLM + sous-chaînes spécialisées + `RunnableBranch`.
2. **with_structured_output + Literal** = garantie que la sortie de classification est dans un ensemble fermé, donc routable sans validation supplémentaire.
3. **RunnablePassthrough.assign(...)** enrichit le dict de contexte au fil de la pipeline, sans perdre les clés précédentes.
4. Le tout reste un **seul Runnable** : `router.invoke`, `router.batch`, `router.astream` fonctionnent — et LangSmith trace l'arbre complet en un seul run.
5. Surcoût : **2 appels LLM par requête** (classification + réponse), au lieu d'un seul avec un prompt méga-universel. Acceptable pour gagner en qualité, mesurable dans LangSmith.

10 Async avec `asyncio.gather` vs `RunnableParallel`

Exercice bonus 5. La voie async pure et sa comparaison avec `RunnableParallel`

Réécris la démo multi-niveaux en **async** avec `await chain.ainvoke(...)` et `asyncio.gather(...)`. Compare les durées avec la version `RunnableParallel` (synchrone et asynchrone).

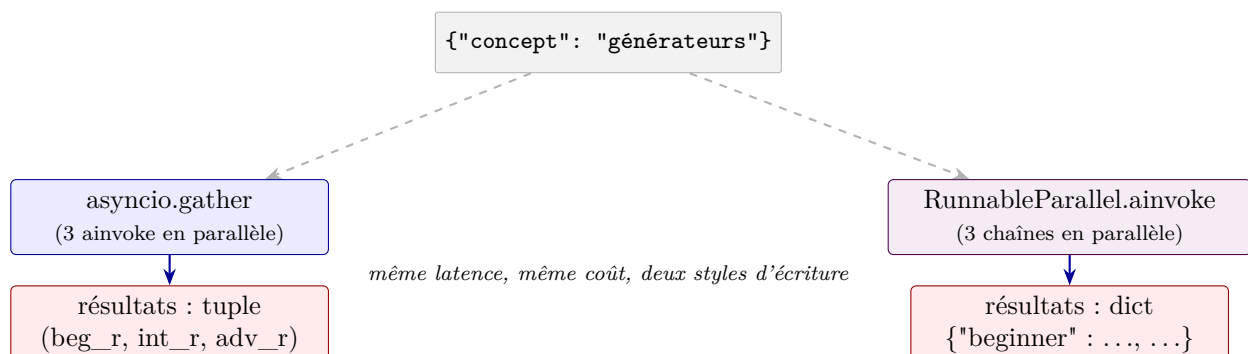
Contrainte : utiliser `asyncio.run()` comme point d'entrée pour la fonction `async`, et `await` sur `.ainvoke()` (la variante asynchrone de `.invoke()`).

Objectifs pédagogiques

- Connaître les méthodes `.ainvoke()`, `.abatch()`, `.astream()` — les variantes async de tous les `Runnable`.
- Comprendre que `RunnableParallel` utilise `asyncio` **sous le capot** : la version async et la version sync donnent des temps quasi identiques.
- Choisir entre `asyncio.gather` (contrôle explicite) et `RunnableParallel` (composition implicite).

10.1 Solution

10.1.1 Architecture



Les deux approches s'appuient sur le **même runtime asyncio** : event loop unique, 3 coroutines en concurrence, attente non bloquante. La différence est purement stylistique.

10.1.2 Le code complet

```
1 import asyncio
2 import time
3 from langchain_core.runnables import RunnableParallel
4 from phase1_explain import explain_chain_for_level
5
6
7 async def exercise_5() -> None:
8     concept      = "les générateurs Python"
9     beginner     = explain_chain_for_level("débutant")
10    intermediate  = explain_chain_for_level("intermédiaire")
11    advanced      = explain_chain_for_level("avancé")
12
13    # --- Version asyncio.gather pure ---
14    t0 = time.perf_counter()
15    beg_r, int_r, adv_r = await asyncio.gather(
16        beginner.ainvoke({"concept": concept}),
17        intermediate.ainvoke({"concept": concept}),
18        advanced.ainvoke({"concept": concept}),
19    )
20    gather_time = time.perf_counter() - t0
21    print(f"asyncio.gather      : {gather_time:.2f}s")
22
23    # --- Version RunnableParallel avec .ainvoke (async aussi) ---
24    parallel = RunnableParallel(
25        beginner=beginner, intermediate=intermediate, advanced=advanced,
26    )
27    t0 = time.perf_counter()
28    results = await parallel.ainvoke({"concept": concept})
29    parallel_time = time.perf_counter() - t0
30    print(f"RunnableParallel    : {parallel_time:.2f}s")
31
32    # --- Version synchrone pour référence ---
33    t0 = time.perf_counter()
34    parallel.invoke({"concept": concept})
35    sync_time = time.perf_counter() - t0
36    print(f"Sync (RunnablePar): {sync_time:.2f}s")
37
38
39 # Point d'entrée : asyncio.run lance la coroutine
40 if __name__ == "__main__":
41     asyncio.run(exercise_5())
```

10.1.3 Explications pas à pas

Étape 1 — Le préfixe a : la version async de tout

LCEL définit une variante async pour chaque méthode synchrone d'un Runnable :

Synchrone	Asynchrone
.invoke(input)	.ainvoke(input)
.batch(inputs)	.abatch(inputs)
.stream(input)	.astream(input)

Les versions async retournent des **coroutine**, à attendre avec **await** dans un contexte **async def**.

Étape 2 — `asyncio.gather` pur

`asyncio.gather(*coros)` prend N coroutines et les exécute en **concurrency** sur l'évent loop. **await** sur le résultat attend que toutes terminent et renvoie un **tuple** des résultats dans l'ordre des arguments :

```
1 beg_r, int_r, adv_r = await asyncio.gather(  
2     beginner.ainvoke({"concept": concept}),    # → beg_r  
3     intermediate.ainvoke({"concept": concept}), # → int_r  
4     advanced.ainvoke({"concept": concept}),    # → adv_r  
5 )
```

C'est l'équivalent *asynchrone* d'un `ThreadPoolExecutor` : le main thread ne fait rien pendant que les 3 appels HTTP sont en vol.

Étape 3 — `RunnableParallel` utilise `asyncio` sous le capot

Quand on appelle `parallel.invoke(...)` (synchrone), LCEL crée en interne un event loop, lance les branches comme des coroutines, et attend le tout. Quand on appelle `parallel.ainvoke(...)`, on réutilise l'évent loop existant — plus efficace, surtout dans une app web déjà en async.

Conséquence pratique : `parallel.invoke()` et `await parallel.ainvoke()` donnent des temps identiques à la milliseconde près. Le choix est dicté par le contexte d'appel (synchrone ou asynchrone), pas par la performance.

Étape 4 — Quand utiliser `gather` vs `RunnableParallel`

<code>asyncio.gather</code>	<code>RunnableParallel</code>
Bibliothèque standard Python.	Primitive LCEL.
Contrôle explicite du flot de coroutines.	Composition déclarative dans une chaîne.
Bien pour du code applicatif : routes FastAPI, boucle de chat, jobs.	Bien pour la <i>composition</i> dans un <code>Runnable</code> (qui sera lui-même streamé, batché, tracé).
Pas de tracing LangSmith par défaut (faut envelopper manuellement).	Tracing LangSmith automatique.

*Règle pratique : `RunnableParallel` dans les **chaînes** (parce que c'est composable et tracé) ; `asyncio.gather` dans le **code applicatif** (parce que c'est explicite et standard Python).*

Étape 5 — Le point d'entrée `asyncio.run`

Une fonction **async def** ne s'exécute pas avec un appel direct : elle renvoie une coroutine non encore lancée. `asyncio.run(coro)` crée un event loop, exécute la coroutine, et ferme proprement.

```
1 async def exercise_5() -> None:  
2     ...                               # contient des await  
3  
4 # Dans le main, on emballe avec asyncio.run  
5 if __name__ == "__main__":
```

```
6 asyncio.run(exercise_5())
```

Sortie de l'exécution

```
=====
EXERCICE 5 - Async avec asyncio.gather vs RunnableParallel
=====
asyncio.gather : 8.74s
RunnableParallel : 9.61s
Sync (RunnablePar) : 8.42s
Échantillon (les générateurs Python, débutant) :
Un générateur est une fonction spéciale qui produit des valeurs une à la fois, au lieu de tout calculer d'un coup.
Imagine une usine qui fabrique des bonbons : au lieu de fabriquer 1000 bonbons et les...
=====
[OK] Tous les exercices de la Phase 1 sont résolus.
=====
```

10.2 Vérification dans LangSmith

Les deux versions (`gather` et `RunnableParallel.invoke`) produisent des arbres très similaires dans LangSmith. La différence :

- `RunnableParallel.invoke` : un *seul* run racine `RunnableParallel` avec 3 enfants (un par branche).
- `asyncio.gather` avec 3 `.invoke()` séparés : **3 runs racines distincts**, chacun étant la trace d'un `RunnableSequence`. Pas d'agrégation automatique.

Pour avoir un seul run agrégé en `gather`, il faut envelopper manuellement avec `trace` de `langsmith` :

```
1 from langsmith import trace
2
3 with trace("multi-level parallel"):
4     beg_r, int_r, adv_r = await asyncio.gather(...)
```

*La traçabilité **automatique** est un argument fort pour `RunnableParallel`. Dès qu'on sort de LCEL, le tracing devient manuel.*

Récapitulatif

Ce qu'il faut retenir :

1. Tout `Runnable` expose une variante *async* : `.invoke`, `.abatch`, `.astream`. Le préfixe *a* = *async*.
2. `asyncio.gather(*coros)` lance *N* coroutines en concurrence et renvoie un tuple des résultats dans l'ordre.
3. `RunnableParallel` utilise `asyncio` sous le capot — `.invoke()` et `await .invoke()` ont les mêmes performances.
4. Règle pratique : `RunnableParallel` dans les chaînes (pour la composition et le tracing), `asyncio.gather` dans le code applicatif (pour le contrôle explicite).
5. `asyncio.run(coro)` est le point d'entrée pour appeler une coroutine depuis un contexte synchrone (script, `if __name__ == "__main__"`).